FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

INFORME DE PROYECTO FINAL

SISTEMA DE GESTIÓN DE LABORATORIOS LABTRACK

Autores:

BOÑON FERNÁNDEZ, Josue Samuel *(% participación)*

HERRERA RUITON, Jhenifer *(% participación)*

QUILICHE CABANILLA, Carlos Miguel *(% participación)*

RAICO MOYA, Frank Jhancarlos *(% participación)*

VILLANUEVA COJAL, Luis Oswaldo *(% participación)*

Curso:

Calidad y Pruebas de Software

Docente del Curso:

Mark Anthony Arribasplata Palomino

Cajamarca – Perú

2026-2

ÍNDICE DE CONTENIDO

RESUMEN 7
Abstract 7
INTRODUCCIÓN 9
Motivación del proyecto 9
Propuestas 9
Alcance y limitaciones del informe 9
ANÁLISIS DEL PROBLEMA 11
Antecedentes 11
Descripción de la organización 11
Identificación y formulación del problema 11
Definición de objetivos 12
BASES TEÓRICAS 13
Calidad de producto de software 13
Pruebas de software y niveles de verificación 13
Análisis estático y deuda técnica 13
Pruebas de rendimiento: carga, estrés y percentiles 13
Automatización de pruebas de interfaz 14
Accesibilidad web 14
Seguridad de aplicaciones 14
HERRAMIENTAS DE INGENIERÍA 15
Marcos de trabajo y estándares adoptados 15
Selección de herramientas de software 15
Lista de requerimientos 16
GENERACIÓN DE SOLUCIONES 18
Visión general de la solución 18
Diagrama de clases 18
Modelo y diagrama de base de datos 18
Script de base de datos 19
Diagrama de paquetes y modelo arquitectónico 20
Capturas del software en implementación 20
METODOLOGÍA DE DESARROLLO 23
Definición de los requerimientos del negocio 23
Historias de usuario, criterios de aceptación y trazabilidad 23
Casos de prueba 24
Definición de las reglas del negocio 25

Implementación del proyecto 26
Pruebas unitarias y de integración 26
Pruebas funcionales con Selenium 30
Documentación de la refactorización 32
Análisis estático: SonarQube antes y después 34
Pruebas de carga y estrés con Apache JMeter 38
Accesibilidad e inclusión 43
RECOMENDACIONES FINALES 44
Conclusiones 44
Recomendaciones 44
REFERENCIAS O BIBLIOGRAFÍA 46
ANEXOS – RECURSOS 47
Anexo A. Matriz completa de casos de prueba y trazabilidad 48
Anexo B. Uso de agentes de IA: prompts y skills 51
Anexo C. Índice de artefactos y evidencias 54
Anexo D. Guía de reproducción y credenciales de demostración 55
NOTAS ACLARATORIAS 56

ÍNDICE DE TABLAS

Tabla 1. Indicadores consolidados de la auditoría de calidad de LabTrack. 8
Tabla 2. Estándares y marcos de referencia utilizados en la auditoría. 15
Tabla 3. Herramientas de ingeniería y versiones utilizadas. 15
Tabla 4. Requerimientos funcionales y estado verificado en el repositorio. 16
Tabla 5. Requerimientos no funcionales y criterio de aceptación. 17
Tabla 6. Modelo de datos implementado en backend/prisma/schema.prisma..... 19
Tabla 7. Historias de usuario declaradas, criterios de aceptación y estado de evidencia. 24
Tabla 8. Distribución de casos de prueba por historia de usuario y origen. 25
Tabla 9. Casos de borde sugeridos por IA con validación humana pendiente. 25
Tabla 10. Reglas de negocio implementadas y su verificación. 25
Tabla 11. Evolución de las pruebas unitarias y de la cobertura de código. 26
Tabla 12. Corridas reales de la suite Selenium y resultado obtenido. 31
Tabla 13. Flujos críticos automatizados con Selenium y evidencia asociada. 31
Tabla 14. Hallazgos de SonarQube que concentraban la deuda técnica inicial..... 33
Tabla 15. Backlog de refactorización con estado y evidencia de cierre. 34
Tabla 16. Comparación de métricas de SonarQube antes y después de la refactorización. 35
Tabla 17. Matriz consolidada de los pilares de calidad de SonarQube. 38
Tabla 18. Resumen global de la prueba de carga con 50 usuarios concurrentes. 38
Tabla 19. Detalle por transacción de la prueba de carga. 39
Tabla 20. Resumen global de la prueba de estrés con 150 usuarios concurrentes. 40
Tabla 21. Detalle por transacción de la prueba de estrés. 41
Tabla 22. Resultados de la revisión automatizada de accesibilidad con axe-core..... 43
Tabla 23. Skills y mecanismos empleados durante el desarrollo asistido. 51
Tabla 24. Tareas asignadas a cada agente y alcance de archivos. 51
Tabla 25. Artefactos, ubicación y naturaleza de la evidencia..... 54

ÍNDICE DE FIGURAS

Figura 1. Arquitectura de tres capas de LabTrack y ubicación de las herramientas de calidad. 18
Figura 2. Catálogo de equipos visible para una sesión de estudiante. 21
Figura 3. Reserva registrada en estado pendiente desde el catálogo. 21
Figura 4. Solicitud aprobada por el administrador. 22
Figura 5. Devolución tardía con penalización aplicada. 22
Figura 6. Pirámide de pruebas aplicada: base de pruebas unitarias, interfaz y cima de flujos E2E. 23
Figura 7. Salida de Vitest del backend después de incorporar los casos de límites (53 pruebas aprobadas). 27
Figura 8. Salida de Vitest del frontend después de incorporar los casos de límites (48 pruebas aprobadas). 29
Figura 9. Ejecución contra mocks defectuosos: la suite detecta el fallo y termina con código de salida 1. 29
Figura 10. Reporte HTML de cobertura del backend generado con Istanbul/V8. 30
Figura 11. Comparativa de cobertura entre backend y frontend medida con Istanbul/V8. 30
Figura 12. Consola de la primera corrida: 4 pruebas aprobadas y 3 fallidas por la regresión de selectores. 32
Figura 13. Consola de la corrida final: 7 pruebas aprobadas y 0 fallidas. 32
Figura 14. SonarQube, estado ANTES reproducido: 44 incidencias, confiabilidad B y seguridad C. 36
Figura 15. Listado completo de las 44 incidencias del estado ANTES con regla y archivo. 36
Figura 16. SonarQube, estado DESPUÉS: 0 incidencias y calificaciones A/A/A. 37
Figura 17. Bandeja de incidencias vacía después de la refactorización. 37
Figura 18. Comparativo real de incidencias de SonarQube antes y después. 38
Figura 19. Dashboard de JMeter de la prueba de carga: 0.00 % de error sobre 79 424 peticiones. 39
Figura 20. Tiempo de respuesta (promedio y percentil 95) durante la prueba de carga con 50 usuarios. 40
Figura 21. Distribución de percentiles de la prueba de carga con 50 usuarios. 40
Figura 22. Throughput y usuarios concurrentes durante la prueba de carga con 50 usuarios. 40
Figura 23. Dashboard de JMeter de la prueba de estrés: 0.00 % de error sobre 113 600 peticiones. 41
Figura 24. Tiempo de respuesta (promedio y percentil 95) durante la prueba de estrés con 150 usuarios. 42
Figura 25. Distribución de percentiles de la prueba de estrés con 150 usuarios. 42
Figura 26. Throughput y usuarios concurrentes durante la prueba de estrés con 150 usuarios. 42

ÍNDICE DE ANEXOS

Anexo A. Matriz completa de casos de prueba y trazabilidad 48
Anexo B. Uso de agentes de IA: prompts y skills 51
Anexo C. Índice de artefactos y evidencias 54
Anexo D. Guía de reproducción y credenciales de demostración 55

RESUMEN

El presente informe documenta la auditoría de calidad aplicada al Sistema de Gestión de Laboratorios LabTrack, una aplicación web para reservar, aprobar y devolver equipos de los laboratorios de cómputo, electrónica y redes de la Universidad Privada del Norte. El trabajo se organizó en cinco frentes verificables: análisis estático con SonarQube, pruebas unitarias y de integración con Vitest, automatización funcional de extremo a extremo con Selenium, pruebas de rendimiento con Apache JMeter y revisión de accesibilidad con axe-core. Cada resultado se contrastó contra un artefacto del repositorio; las cifras que no pudieron sustentarse se marcan expresamente como pendientes.

El alcance declarado por el equipo comprende doce historias de usuario (HU-01, HU-02, HU-03, HU-04, HU-05, HU-06, HU-07, HU-08, HU-11, HU-13, HU-14 y HU-15). La verificación del repositorio confirma evidencia completa, con código y pruebas, para cinco de ellas; las siete restantes se reportan como brechas y no se contabilizan como funcionalidad implementada. En el plano técnico, SonarQube pasó de 44 incidencias abiertas (1 bug, 2 vulnerabilidades y 41 code smells; confiabilidad B y seguridad C) a 0 incidencias (A/A/A). Las pruebas unitarias crecieron de 93 a 101 casos aprobados, con una cobertura de 95.71 % en el backend y 90.26 % en el frontend. La suite Selenium aprobó 4 de 7 flujos en su primera corrida y 7 de 7 en la corrida final, después de corregir una regresión de selectores provocada por la refactorización de accesibilidad S6819. Las pruebas de carga (50 usuarios) y estrés (150 usuarios) registraron 0.00 % de errores, con percentiles 95 de 48 ms y 151 ms respectivamente. La revisión de accesibilidad detectó únicamente fallas de contraste de color, todavía pendientes de corrección.

Palabras clave: calidad de software; pruebas automatizadas; análisis estático; pruebas de rendimiento; accesibilidad web.

Abstract

This report documents the quality audit of LabTrack, a web system for managing laboratory equipment reservations at Universidad Privada del Norte. The audit covered static analysis with SonarQube, unit and integration testing with Vitest, end-to-end automation with Selenium, performance testing with Apache JMeter and accessibility review with axe-core. SonarQube went from 44 open issues to zero (A/A/A); unit tests increased from 93 to 101 with 95.71 % backend and 90.26 % frontend coverage; the Selenium suite improved from 4 of 7 passing flows to 7 of 7 after fixing a selector regression caused by the S6819 accessibility refactor; load (50 users) and stress (150

users) tests reported 0.00 % errors with p95 values of 48 ms and 151 ms. Accessibility review found only color-contrast violations, which remain open. Seven of the twelve declared user stories lack verifiable evidence in the repository and are reported as gaps.

Keywords: software quality; automated testing; static analysis; performance testing; web accessibility.

Tabla 1**.** Indicadores consolidados de la auditoría de calidad de LabTrack.

Dimensión	Indicador	Antes	Después	Criterio	Veredicto
Pruebas unitarias	Casos aprobados	93	101 (+8)	100 % en verde	Cumple
Cobertura backend	Sentencias / ramas	95.71 % / 85.00 %	95.71 % / 85.00 %	≥ 80 %	Cumple
Cobertura frontend	Sentencias / ramas	90.26 % / 81.69 %	90.26 % / 81.69 %	≥ 80 %	Cumple
SonarQube	Bugs / vulnerabilidades / smells	1 / 2 / 41	0 / 0 / 0	0 incidencias	Cumple
Confiabilidad y seguridad	Ratings SonarQube	B / C	A / A	A / A	Cumple
Selenium E2E	Flujos aprobados	4 de 7	7 de 7	7 de 7	Cumple tras corrección
JMeter carga	Muestras / error / p95	—	79 424 / 0.00 % / 48 ms	p95 < 3000 ms	Cumple
JMeter estrés	Muestras / error / p95	—	113 600 / 0.00 % / 151 ms	p95 < 5000 ms	Cumple
Accesibilidad	Violaciones axe-core	—	1 en login, 17 en catálogo	0 violaciones	Pendiente
Trazabilidad	HU con evidencia completa	5 de 12	5 de 12	12 de 12	Brecha declarada

INTRODUCCIÓN

Motivación del proyecto

La gestión física de los laboratorios universitarios suele apoyarse en bitácoras de papel o en hojas de cálculo que no se sincronizan entre sí. Esa práctica produce tres problemas recurrentes: colisiones de horario sobre un mismo equipo, pérdida de la trazabilidad del responsable del préstamo y devoluciones fuera de plazo que no generan consecuencia alguna. La digitalización del ciclo completo —reserva, aprobación, entrega, devolución y penalización— permite sustituir la confianza informal por reglas verificables y por un registro de auditoría consultable.

La motivación académica del trabajo es complementaria. Un sistema que funciona no es necesariamente un sistema de calidad: para afirmarlo hace falta evidencia empírica sobre su corrección funcional, su comportamiento bajo concurrencia, la salud de su código y su accesibilidad. Este informe reúne esa evidencia y, sobre todo, distingue con claridad lo que fue ejecutado de lo que quedó pendiente.

Propuestas

LabTrack se propone como una aplicación web con tres perfiles de uso: el estudiante reserva equipos y consulta sus préstamos; el administrador aprueba o rechaza solicitudes y registra devoluciones; el sistema aplica automáticamente la penalización cuando la devolución ocurre fuera del horario comprometido. La arquitectura es desacoplada: una interfaz de página única en React, una API REST en NestJS y una base de datos relacional PostgreSQL administrada con Prisma ORM.

Sobre esa base, la propuesta de aseguramiento de la calidad combina cuatro mecanismos. El primero es el análisis estático continuo, que detecta deuda técnica antes de que llegue a producción. El segundo es la pirámide de pruebas: unitarias e integración en la base, funcionales de extremo a extremo en la cima. El tercero es la medición del rendimiento con perfiles de carga y estrés. El cuarto es la revisión de accesibilidad, entendida como requisito de calidad y no como un añadido cosmético.

Alcance y limitaciones del informe

- El código del estado ANTES de SonarQube es reproducido, no histórico: el repositorio tiene un único commit y no conserva una versión previa a la refactorización. Los 44 defectos se reinyectaron en una copia de trabajo y así se documenta en todo el informe.

- Testomat.io, TestLink y Katalon Studio no se ejecutaron. Se entrega la plantilla de importación y el archivo CSV, sin fabricar capturas de herramientas que no se abrieron.
- WAVE no se ejecutó. La revisión de accesibilidad se realizó con axe-core 4.10.2, el mismo motor de la extensión, sobre las páginas de login y catálogo.
- Siete de las doce historias de usuario declaradas no tienen evidencia verificable de implementación completa; se reportan como brechas y no como funcionalidades terminadas.
- Los datos organizacionales que la plantilla solicita (historia de la institución, tamaño del área, indicadores internos) no fueron provistos por el equipo y se dejan como marcadores visibles.

ANÁLISIS DEL PROBLEMA

Antecedentes

La plantilla institucional incluye antecedentes sobre automatización de buzones tributarios de SUNAT y SUNAFIL que pertenecen a otro caso de estudio y no guardan relación con LabTrack. Para no atribuir a este proyecto investigaciones que no se consultaron, esos párrafos se retiran y se deja el espacio para que el equipo incorpore sus propias fuentes.

[COMPLETAR POR EL EQUIPO: antecedentes de investigación del proyecto, con autor, año, objetivo, metodología y resultado verificable; citar solo fuentes efectivamente leídas y enlazadas en las referencias]

Como marco de referencia técnico —no como antecedente de investigación— este informe se apoya en estándares públicos: la familia ISO/IEC 25010 para el modelo de calidad de producto (International Organization for Standardization [ISO], 2011), el syllabus del ISTQB para la terminología de pruebas (International Software Testing Qualifications Board [ISTQB], 2023) y las WCAG 2.1 del W3C para accesibilidad (World Wide Web Consortium [W3C], 2018).

Descripción de la organización

El caso de estudio se sitúa en los laboratorios de la carrera de Ingeniería de Sistemas Computacionales de la Universidad Privada del Norte, sede Cajamarca. El equipo del curso Calidad y Pruebas de Software desarrolló LabTrack como proyecto formativo y lo sometió a una auditoría de calidad completa durante el periodo 2026-2.

[COMPLETAR POR EL EQUIPO: descripción formal de la organización (reseña institucional, población de usuarios, cantidad de laboratorios y equipos, procesos internos de préstamo y responsables), usando únicamente información oficial o proporcionada por la institución]

Identificación y formulación del problema

El proceso manual presenta cuatro fallas observables. Primero, la disponibilidad de un equipo no se conoce en tiempo real, de modo que dos estudiantes pueden creer que tienen derecho al mismo recurso en la misma franja. Segundo, la aprobación depende de la presencia física del responsable, lo que concentra la operación en horarios de oficina. Tercero, la devolución tardía no deja rastro ni consecuencia, por lo que el incentivo a devolver a tiempo es débil. Cuarto, no existe un registro único que permita responder quién tuvo un equipo, cuándo y en qué estado lo devolvió.

Formulación: ¿en qué medida un sistema web con reglas de negocio explícitas, trazabilidad de eventos y controles automatizados de calidad puede sustituir el proceso manual de préstamo de equipos, cumpliendo criterios verificables de corrección funcional, rendimiento, mantenibilidad y accesibilidad?

Definición de objetivos

Objetivo general

Demostrar, con evidencia empírica y artefactos auditables, que LabTrack satisface los requisitos funcionales y no funcionales definidos para su operación en los laboratorios, y determinar con honestidad qué brechas impiden afirmar una certificación plena.

Objetivos específicos

- Diseñar y ejecutar una matriz de casos de prueba para las historias con evidencia verificable, e identificar las historias declaradas que carecen de sustento en el repositorio.
- Verificar la corrección funcional en tres niveles: integración de API, componentes de interfaz y flujos de extremo a extremo en navegador real.
- Medir el comportamiento no funcional bajo carga sostenida y estrés, y contrastarlo con umbrales de servicio explícitos.
- Diagnosticar la deuda técnica con análisis estático, aplicar una refactorización guiada y comprobar la reducción de incidencias.
- Evaluar la accesibilidad de las pantallas principales y registrar los defectos que permanecen abiertos.
- Consolidar la trazabilidad entre requisitos, casos de prueba, defectos y evidencias, dejando constancia de los límites del alcance.

BASES TEÓRICAS

Calidad de producto de software

La norma ISO/IEC 25010 define un modelo de calidad de producto con ocho características: adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad (ISO, 2011). El modelo es útil porque obliga a evaluar el sistema desde varias dimensiones y no solo desde «funciona o no funciona». En este informe, la adecuación funcional se evalúa con la matriz de casos y Selenium; la eficiencia de desempeño, con JMeter; la fiabilidad y la mantenibilidad, con SonarQube y las pruebas unitarias; la seguridad, con las reglas de análisis estático y el OWASP Top 10; la usabilidad y la accesibilidad, con axe-core y las WCAG 2.1.

Pruebas de software y niveles de verificación

El ISTQB organiza las pruebas por nivel —componentes, integración, sistema y aceptación— y por tipo —funcionales y no funcionales— (ISTQB, 2023). La pirámide de pruebas traduce esa clasificación en una estrategia económica: muchas pruebas rápidas en la base y pocas pruebas lentas en la cima. LabTrack aplica esa idea con 101 pruebas de Vitest en la base, pruebas de componentes con React Testing Library en la interfaz y siete flujos de extremo a extremo con Selenium en la cima. La norma IEEE 829 complementa el marco al definir el contenido esperado de la documentación de pruebas (IEEE, 2008).

Análisis estático y deuda técnica

El análisis estático examina el código sin ejecutarlo y clasifica los hallazgos en bugs (fiabilidad), vulnerabilidades (seguridad) y code smells (mantenibilidad). SonarQube aplica reglas parametrizadas por lenguaje, calcula calificaciones de A a E y estima el esfuerzo de remediación (SonarSource, 2024). Su filosofía «Clean as You Code» prioriza el código nuevo, razón por la cual la puerta de calidad puede aprobar un proyecto que aún conserva deuda antigua. Comprender esa distinción es imprescindible para interpretar correctamente los resultados de este informe.

Pruebas de rendimiento: carga, estrés y percentiles

Una prueba de carga verifica el cumplimiento de los acuerdos de nivel de servicio bajo la demanda esperada; una prueba de estrés lleva el sistema más allá de su dimensionamiento para observar cómo degrada (Apache Software Foundation, 2024). El percentil 95 es la métrica preferida frente al promedio porque revela la latencia de cola: un promedio bajo puede ocultar un pequeño grupo de usuarios con esperas

enormes. En este trabajo se reportan mediana, p90, p95, p99, máximo, desviación estándar, throughput y tasa de error para cada transacción.

Automatización de pruebas de interfaz

Selenium WebDriver controla un navegador real mediante el protocolo W3C WebDriver y permite automatizar recorridos completos de usuario (Selenium Project, 2024). Su fortaleza es la fidelidad: prueba lo mismo que ve la persona. Su debilidad es la fragilidad de los localizadores, que se rompen cuando cambia el marcado. Esa fragilidad se materializó en este proyecto y se documenta como hallazgo, porque constituye una lección de ingeniería más valiosa que una corrida perfecta.

Accesibilidad web

Las WCAG 2.1 organizan los requisitos de accesibilidad en cuatro principios —perceptible, operable, comprensible y robusto— con niveles A, AA y AAA (W3C, 2018). El nivel AA es el referente habitual para sitios institucionales. La especificación WAI-ARIA advierte que los elementos nativos de HTML deben preferirse a los roles ARIA simulados, porque aportan semántica y comportamiento de teclado de forma gratuita (W3C, 2023). axe-core automatiza parte de la verificación, aunque no sustituye la revisión manual ni las pruebas con usuarios (Deque Systems, 2024).

Seguridad de aplicaciones

El OWASP Top 10 identifica las categorías de riesgo más relevantes para aplicaciones web, entre ellas el fallo de control de acceso, la identificación y autenticación deficientes y la exposición de credenciales (OWASP Foundation, 2021). Las reglas S2068 y S2245 de SonarQube, que detectaron una contraseña embebida y un generador pseudoaleatorio inseguro, se interpretan en este informe a la luz de esa taxonomía.

HERRAMIENTAS DE INGENIERÍA

Marcos de trabajo y estándares adoptados

El equipo no documentó la adopción formal de Scrum, Kanban ni otro marco ágil, por lo que este informe no atribuye uno. Lo que sí puede verificarse es la secuencia de trabajo: definición de requisitos, construcción, verificación automatizada, análisis estático, corrección y nueva verificación. Esa secuencia se corresponde con un ciclo de aseguramiento iterativo y con los estándares que se resumen a continuación.

[COMPLETAR POR EL EQUIPO: marco de trabajo y cronograma efectivamente utilizados (sprints, iteraciones o fases), con fechas y responsables; si se usó Scrum u otro marco, adjuntar la evidencia de su aplicación]

Tabla 2**.** Estándares y marcos de referencia utilizados en la auditoría.

Estándar o marco	Aporte al proyecto	Dónde se aplica en este informe
ISO/IEC 25010:2011	Modelo de calidad de producto en ocho características.	Estructura de la evaluación no funcional y de las conclusiones.
ISTQB CTFL 4.0	Vocabulario y niveles de prueba.	Diseño de la matriz de casos y de la pirámide de pruebas.
IEEE 829-2008	Contenido de la documentación de pruebas.	Estructura de la matriz y de los registros de ejecución.
WCAG 2.1 nivel AA	Criterios de accesibilidad verificables.	Revisión con axe-core y registro del defecto de contraste.
WAI-ARIA 1.2	Uso correcto de roles y preferencia por HTML nativo.	Hallazgos S6819, S1082 y S6847.
OWASP Top 10:2021	Taxonomía de riesgos de seguridad.	Interpretación de S2068 y S2245.
Sonar way	Perfil de reglas y puerta de calidad.	Análisis estático antes y después.

Selección de herramientas de software

Las herramientas se eligieron por tres criterios: gratuidad para uso académico, disponibilidad en el entorno local del equipo y capacidad de producir evidencia exportable. La tabla siguiente registra las versiones realmente utilizadas, tomadas de los archivos de configuración del repositorio y de los reportes de ejecución.

Tabla 3**.** Herramientas de ingeniería y versiones utilizadas.

Herramienta	Versión	Propósito	Evidencia generada
SonarQube Community Build	26.10.0.133078	Análisis estático de backend y frontend.	Capturas reales, JSON de API y tabla comparativa.
SonarScanner CLI	8.3.1.16491	Envío de métricas al servidor SonarQube.	Registros de escaneo sin credenciales.
OpenJDK	21	Requisito de SonarQube y del scanner.	Entorno de ejecución.
Node.js / npm	Entorno del proyecto	Ejecución de backend, frontend y pruebas.	Salidas de consola.
NestJS	11.1.6	Framework de la API REST.	Código fuente y pruebas de integración.
React	18	Interfaz de usuario de página única.	Componentes y pruebas de interfaz.
Vite	6.0.7	Servidor de desarrollo y compilación del frontend.	Frontend compilado en el puerto 4173.

Herramienta	Versión	Propósito	Evidencia generada
Prisma ORM	6.16.2	Modelo y acceso a datos PostgreSQL.	Esquema y migraciones.
PostgreSQL	18	Motor de base de datos relacional.	Base de pruebas labtrack_test.
Vitest	3.2.4	Pruebas unitarias e integración; cobertura V8.	101 pruebas y reportes de cobertura.
Supertest	7.1.4	Pruebas HTTP de la API NestJS.	Casos de integración del backend.
React Testing Library	Incluida en el proyecto	Pruebas de componentes sobre jsdom.	Casos de interfaz del frontend.
Selenium WebDriver	4.27 (^4.27.0)	Automatización E2E en Chrome headless.	JUnit XML y capturas de pantalla.
Apache JMeter	5.6.3	Carga (50 usuarios) y estrés (150 usuarios).	Dashboard HTML, summary.json y gráficos.
axe-core	4.10.2	Revisión automatizada de accesibilidad.	JSON con violaciones por página.
python-docx / python-pptx	1.2.0 / versión instalada	Generación de informe y presentación.	DOCX, PPTX y PDF exportados.

El hardware utilizado no quedó registrado en los artefactos del proyecto. Dado que las cifras de rendimiento dependen del equipo, se deja constancia expresa de esa omisión.

[COMPLETAR POR EL EQUIPO: características del hardware de las pruebas de rendimiento (CPU, memoria, disco, sistema operativo y versión de Chrome) para permitir la reproducción exacta]

Lista de requerimientos

Los requerimientos funcionales se derivaron de las historias declaradas y se contrastaron con el código. La columna de estado distingue lo implementado, lo implementado parcialmente y lo declarado sin sustento; esa distinción es la que evita presentar como terminado lo que solo está enunciado.

Tabla 4. Requerimientos funcionales y estado verificado en el repositorio.

ID	Requerimiento funcional	Estado verificado	Evidencia principal
RQ-01	Autenticación con código y contraseña, emisión de JWT y cierre de sesión.	Implementado	auth.controller.ts, auth.service.ts, LoginPage.tsx
RQ-02	Bloqueo temporal tras tres intentos fallidos.	Implementado (5 min; el criterio declara 15)	auth.service.ts, auth.spec.ts
RQ-03	Catálogo de equipos con filtros por categoría y estado.	Implementado	equipment.service.ts, CatalogPage.tsx
RQ-04	Registro de reservas con validación de horario, duración y solapamiento.	Implementado	reservations.service.ts, reservation-boundaries.extra.spec.ts
RQ-05	Aprobación y rechazo administrativo con motivo obligatorio.	Implementado	admin.service.ts, AdminPendingPage.tsx
RQ-06	Devolución con penalización automática por retraso.	Implementado (sin plazo persistido)	admin.service.ts, AdminReturnsPage.tsx
RQ-07	Bitácora de auditoría de eventos del sistema.	Implementado	AuditLog en schema.prisma, audit service
RQ-08	Panel administrativo con contadores de gestión.	Parcial (sin gráficos ni tasa de puntualidad)	admin.service.ts, AdminPendingPage.tsx
RQ-09	Carnet digital QR del estudiante.	Declarado, sin evidencia	Sin archivos ni pruebas
RQ-10	CRUD administrativo de equipos con fotos y QR.	Parcial (solo consultas)	equipment.controller.ts

ID	Requerimiento funcional	Estado verificado	Evidencia principal
RQ-11	Levantamiento de sanciones con justificación.	Declarado, sin evidencia	Sin endpoint ni interfaz
RQ-12	Acta oficial de préstamo en PDF.	Declarado, sin evidencia	Sin generador ni ruta

Tabla 5**.** Requerimientos no funcionales y criterio de aceptación.

ID	Requerimiento no funcional	Criterio de aceptación	Resultado
RNF-01	Seguridad de credenciales y sesiones.	Contraseñas con bcrypt; sin secretos embebidos; JWT firmado.	Cumple tras refactorización
RNF-02	Eficiencia de desempeño bajo carga.	p95 < 3000 ms y error ≤ 1 % con 50 usuarios.	Cumple: 48 ms y 0.00 %
RNF-03	Eficiencia de desempeño bajo estrés.	p95 < 5000 ms y error ≤ 5 % con 150 usuarios.	Cumple: 151 ms y 0.00 %
RNF-04	Mantenibilidad estática.	Cero bugs, vulnerabilidades y code smells.	Cumple: 0/0/0
RNF-05	Cobertura de pruebas.	≥ 80 % de sentencias en backend y frontend.	Cumple: 95.71 % y 90.26 %
RNF-06	Accesibilidad.	WCAG 2.1 AA sin violaciones automatizadas.	Parcial: contraste pendiente
RNF-07	Compatibilidad de navegadores.	Ejecución E2E en Chrome, Firefox y Edge.	Parcial: solo Chrome
RNF-08	Trazabilidad y auditoría.	Registro de eventos críticos con actor y detalle.	Cumple: modelo AuditLog

GENERACIÓN DE SOLUCIONES

Visión general de la solución

La solución se organiza en tres capas. La capa de presentación es una SPA en React con Vite y Tailwind CSS, que consume la API mediante fetch y protege las rutas con un componente de guarda. La capa de negocio es una API REST NestJS con módulos de autenticación, equipos, reservas y administración; aplica validación de DTO, guards de rol y transacciones Prisma para las operaciones críticas. La capa de datos es PostgreSQL, modelada con Prisma y sembrada con usuarios y equipos de demostración.



Figura 1. Arquitectura de tres capas de LabTrack y ubicación de las herramientas de calidad.

Diagrama de clases

El repositorio no contiene un diagrama de clases formal. Para no inventar relaciones que el código no declara, se deja el espacio correspondiente y se documenta el modelo de datos real en el apartado siguiente.

[COMPLETAR POR EL EQUIPO: diagrama de clases UML con servicios, controladores, DTO y entidades del dominio, generado a partir de backend/src y frontend/src]

Modelo y diagrama de base de datos

El modelo real se compone de cuatro entidades. User almacena credenciales, rol, estado y control de bloqueo. Equipment registra el inventario con código único, categoría, estado y ubicación. Reservation relaciona usuario y equipo, guarda fecha, hora de inicio y fin, estado, motivo de rechazo, marcas de revisión y devolución, y la

bandera de penalización. AuditLog conserva los eventos con actor, detalle, dirección IP y fecha.

Tabla 6. Modelo de datos implementado en backend/prisma/schema.prisma.

Entidad	Campos principales	Relaciones e índices
User	id, code (único), fullName, email, passwordHash, role, status, failedAttempts, lockedUntil, createdAt	1:N con Reservation y AuditLog; índice por role
Equipment	id, name, inventoryCode (único), category, status, description, location, createdAt	1:N con Reservation; índices por category y status
Reservation	id, userId, equipmentId, date, startTime, endTime, status, rejectionReason, reviewedBy, reviewedAt, returnedAt, penaltyApplied	N:1 con User y Equipment; índices por usuario/fecha, equipo/fecha y estado
AuditLog	id, event, actorCode, detail, ipAddress, userId, createdAt	N:1 opcional con User; índices por evento y fecha

El diagrama entidad-relación no existe como archivo en el repositorio; el esquema Prisma es la fuente de verdad y permite generarlo.

[COMPLETAR POR EL EQUIPO: diagrama entidad-relación exportado desde Prisma o desde una herramienta de modelado, con cardinalidades y claves]

Script de base de datos

El proyecto utiliza Prisma Migrate, de modo que el DDL se genera a partir del esquema. Se reproduce el esquema real, que actúa como script de definición de la base de datos.

```
enum Role { STUDENT ADMIN }
enum UserStatus { ACTIVE PENALIZED }
enum EquipmentStatus { AVAILABLE BORROWED }
enum ReservationStatus { PENDING APPROVED REJECTED RETURNED }

model User {
  id             String   @id @default(cuid())
  code           String   @unique
  fullName       String
  email          String   @default("")
  passwordHash   String
  role           Role     @default(STUDENT)
  status         UserStatus @default(ACTIVE)
  failedAttempts Int      @default(0)
  lockedUntil    DateTime?
  createdAt      DateTime @default(now())
  reservations   Reservation[]
  auditLogs      AuditLog[]
  @@index([role])
}

model Equipment {
  id            String  @id @default(cuid())
  name          String
  inventoryCode String  @unique
  category      String
  status        EquipmentStatus @default(AVAILABLE)
  description   String  @default("")
  location      String  @default("Laboratorio principal")
  createdAt     DateTime @default(now())
  reservations  Reservation[]
  @@index([category]) @@index([status])
}
```

```
model Reservation {
  id              String @id @default(cuid())
  userId          String
  equipmentId     String
  date            DateTime @db.Date
  startTime       String @db.VarChar(5)
  endTime         String @db.VarChar(5)
  status          ReservationStatus @default(PENDING)
  rejectionReason String? @db.VarChar(300)
  reviewedBy      String?
  reviewedAt      DateTime?
  returnedAt      DateTime?
  penaltyApplied  Boolean @default(false)
  createdAt       DateTime @default(now())
  user            User @relation(fields: [userId], references: [id])
  equipment       Equipment @relation(fields: [equipmentId], references: [id])
  @@index([userId, date]) @@index([equipmentId, date]) @@index([status])
}

model AuditLog {
  id        String @id @default(cuid())
  event     String
  actorCode String @default("")
  detail    String @default("") @db.VarChar(300)
  ipAddress String @default("") @db.VarChar(60)
  userId    String?
  createdAt DateTime @default(now())
  user      User? @relation(fields: [userId], references: [id])
  @@index([event]) @@index([createdAt])
}
```

Diagrama de paquetes y modelo arquitectónico

El backend se organiza por módulos de dominio (auth, equipment, reservations, admin) con controlador, servicio y DTO por módulo, más un servicio transversal de auditoría y un módulo Prisma. El frontend se organiza por páginas, componentes reutilizables, contextos de autenticación y notificaciones, y una capa de acceso a API. La figura de arquitectura resume esa distribución y la posición de las herramientas de calidad respecto de cada capa.

[COMPLETAR POR EL EQUIPO: diagrama de paquetes formal con dependencias permitidas y prohibidas entre módulos, si el curso exige el artefacto en notación UML]

Capturas del software en implementación

Las siguientes capturas provienen de la corrida final de Selenium sobre el sistema en funcionamiento. No son maquetas: muestran el navegador real recorriendo el catálogo, el registro de una reserva, la aprobación administrativa y la devolución con penalización.

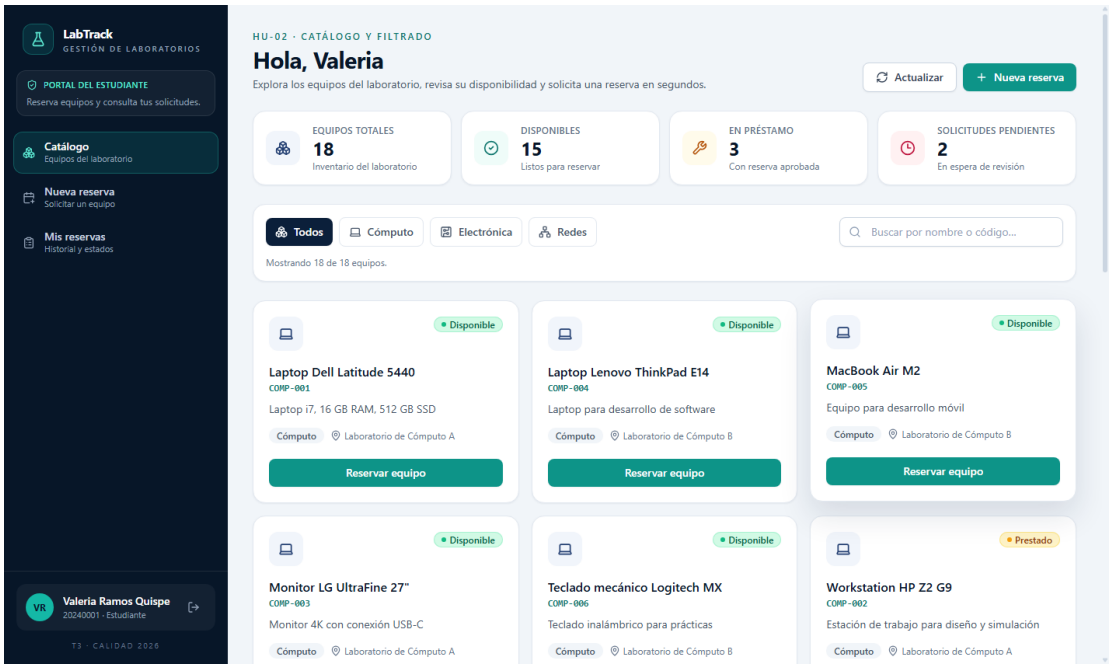


Figura 2. Catálogo de equipos visible para una sesión de estudiante.

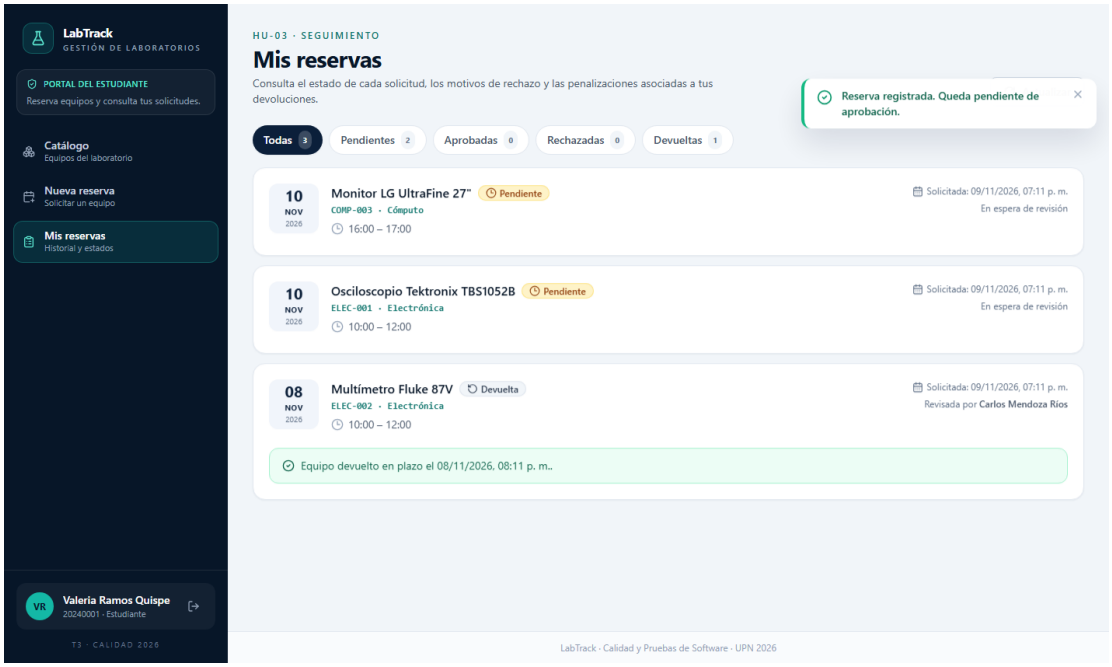


Figura 3. Reserva registrada en estado pendiente desde el catálogo.

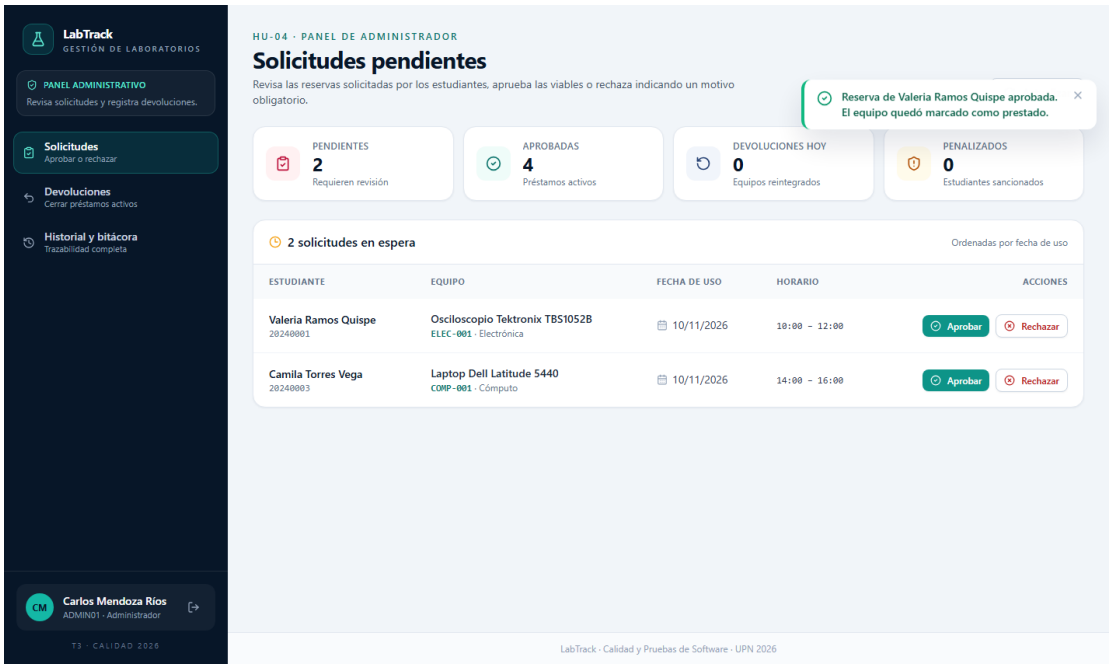


Figura 4. Solicitud aprobada por el administrador.

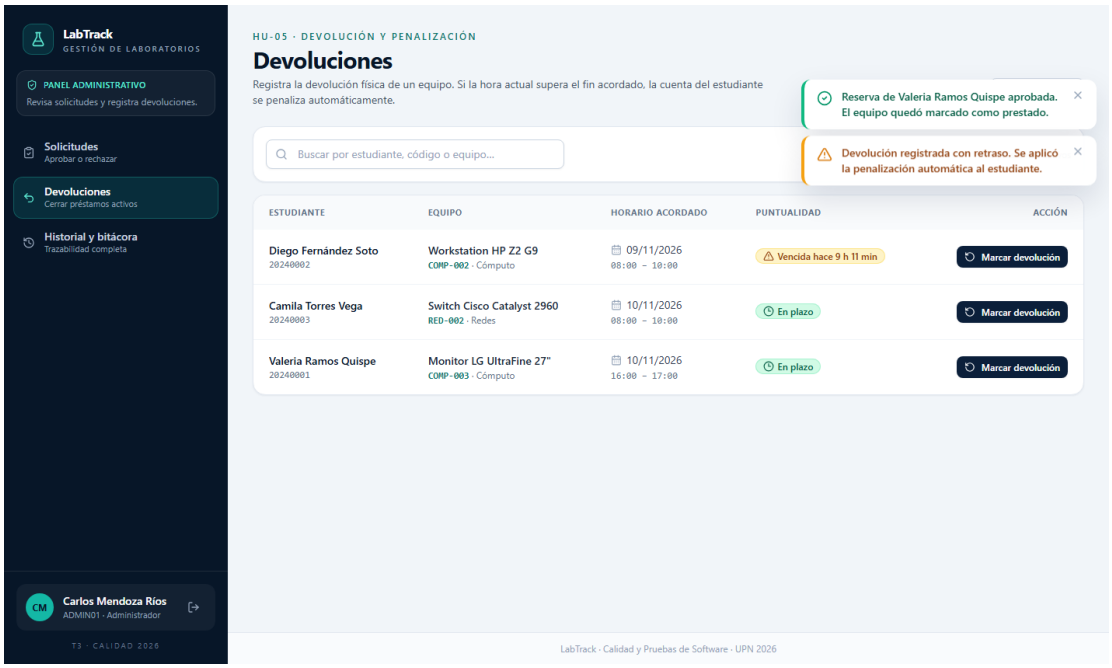


Figura 5. Devolución tardía con penalización aplicada.

METODOLOGÍA DE DESARROLLO

La metodología efectivamente aplicada fue un ciclo de aseguramiento iterativo: se definieron los requisitos y sus criterios, se construyó la solución, se ejecutaron las pruebas en varios niveles, se midió el código con análisis estático, se corrigieron los hallazgos y se volvió a verificar. El ciclo no terminó con «todo verde»: terminó con un balance honesto de lo logrado y de lo que quedó pendiente. Esta sección documenta cada una de esas etapas con sus artefactos.

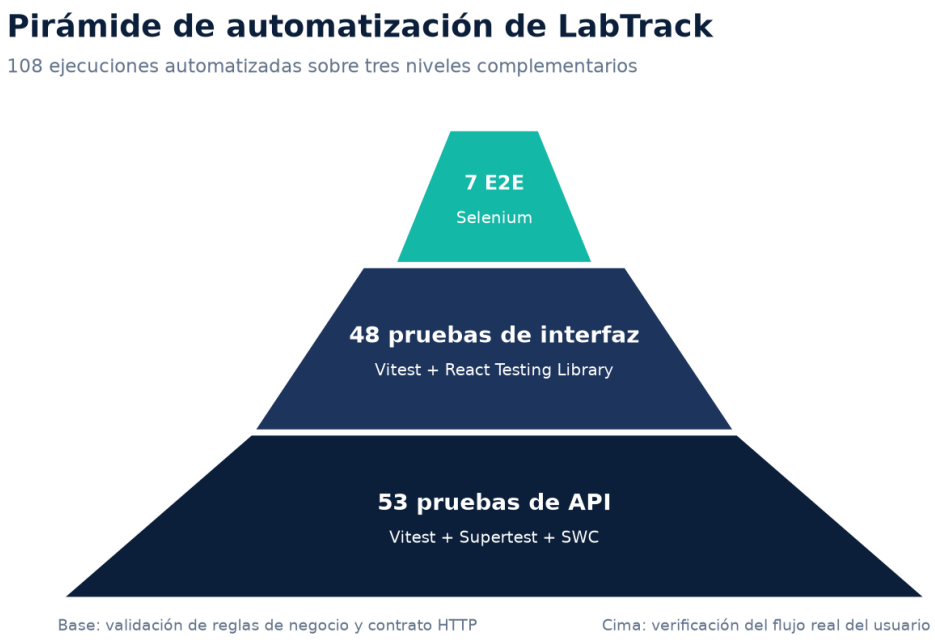


Figura 6**.** Pirámide de pruebas aplicada: base de pruebas unitarias, interfaz y cima de flujos E2E.

Definición de los requerimientos del negocio

Los requerimientos del negocio se expresaron como historias de usuario con criterios de aceptación verificables. La fuente de alcance es el documento de historias implementadas del equipo, que declara doce historias. La verificación se hizo archivo por archivo en backend/src, frontend/src, backend/test y frontend/src/test, sin inferir funcionalidad a partir de la declaración. El resultado es una tabla de trazabilidad con tres estados posibles: implementada con pruebas, implementada sin pruebas y declarada sin evidencia verificable.

Historias de usuario, criterios de aceptación y trazabilidad

La tabla siguiente resume las doce historias declaradas, sus criterios de aceptación y la evidencia encontrada. Las cinco primeras cuentan con código y pruebas automatizadas; las siete restantes carecen de sustento suficiente y se reportan como brechas.

Tabla 7. Historias de usuario declaradas, criterios de aceptación y estado de evidencia.

HU	Descripción	Criterios de aceptación (resumen)	Evidencia en el repositorio	Estado
HU-01	Autenticación segura y control de bloqueos	Código N0 + 8 dígitos; mensaje exacto de credenciales inválidas; bloqueo de 15 minutos tras 3 fallos.	auth.controller.ts, auth.service.ts, login.dto.ts, LoginPage.tsx, AuthContext.tsx; auth.spec.ts, LoginPage.test.tsx	Implementada con pruebas; bloqueo configurado en 5 min y sin regex estricta
HU-02	Catálogo y filtrado de equipos	Filtros exactos Cómputo, Electrónica y Redes; mensaje exacto cuando no hay equipos.	equipment.controller.ts, equipment.service.ts, CatalogPage.tsx; equipment.spec.ts, CatalogPage.test.tsx	Implementada con pruebas
HU-03	Reserva de equipos y reglas críticas	Sin cruces del mismo equipo; una reserva activa por estudiante en el mismo horario; bloquear estudiantes penalizados.	reservations.controller.ts, reservations.service.ts, NewReservationPage.tsx; reservations.spec.ts, reservation-boundaries.extra.spec.ts	Implementada con pruebas
HU-04	Gestión administrativa de solicitudes	Aprobar o rechazar; el rechazo exige motivo.	admin.controller.ts, admin.service.ts, AdminPendingPage.tsx; admin.spec.ts, AdminPendingPage.test.tsx	Implementada con pruebas
HU-05	Devolución y penalización automática	La devolución deja el equipo disponible; el retraso cambia al alumno a penalizado durante 7 días.	admin.controller.ts, admin.service.ts, AdminReturnsPage.tsx; admin.spec.ts, AdminReturnsPage.test.tsx	Implementada con pruebas; no se persiste el plazo de 7 días
HU-06	Carnet digital QR del estudiante	Credencial digital escaneable en ventanilla.	Sin componente, ruta, modelo ni prueba identificable.	Declarada, sin evidencia
HU-07	Gestión integral de equipos (CRUD admin)	Registrar equipos con fotos, especificaciones, ubicación y QR.	Solo consultas de catálogo; sin CRUD, carga de imágenes ni QR.	Declarada, sin evidencia completa
HU-08	Gestión y levantamiento de penalizaciones	Panel admin para levantar sanciones con justificación.	Aplicación automática de la sanción; sin operación de levantamiento.	Declarada, sin evidencia completa
HU-11	Dashboard y métricas de uso	Gráficos de inventario y tasa de puntualidad.	Tarjetas numéricas; sin gráficos ni tasa de puntualidad.	Evidencia parcial
HU-13	Administración y creación de usuarios	Crear alumnos o admins; resetear claves; desbloquear cuentas.	Sin módulo, endpoints ni pruebas de administración de usuarios.	Declarada, sin evidencia
HU-14	Escaneo rápido con cámara o lector QR	Despacho y devolución sin teclear.	Sin integración de cámara, lector ni pantalla de escaneo.	Declarada, sin evidencia
HU-15	Acta oficial de préstamo en PDF	Descargar constancia formal con cláusula de responsabilidad.	Sin generador, endpoint, botón ni plantilla PDF.	Declarada, sin evidencia

Hallazgo de trazabilidad: 7 de 12 historias sin evidencia verificable

La verificación del repositorio confirma cinco historias con código y pruebas (HU-01 a HU-05). HU-06, HU-07, HU-08, HU-11, HU-13, HU-14 y HU-15 no cuentan con evidencia suficiente para afirmar que estén implementadas: no existen rutas, componentes, modelos o pruebas que las sostengan, o solo existe una parte mínima. Además, HU-01 y HU-05 incumplen dos criterios declarados (duración del bloqueo y persistencia del plazo de sanción). El informe no convierte esas brechas en funcionalidades: las registra y las prioriza en el backlog.

La distinción importa porque el sistema puede aprobar sus pruebas y, aun así, no cubrir el alcance prometido. Un informe que solo reportara «7 de 7 flujos E2E aprobados» sería técnicamente cierto y estratégicamente engañoso: esos siete flujos cubren cinco historias, no doce. La conclusión del trabajo incorpora esa diferencia.

Casos de prueba

La matriz completa se organiza por historia y contiene 66 casos funcionales: 61 aprobados para las cinco historias verificables y 5 casos de borde sugeridos por inteligencia artificial con validación humana pendiente. Cada caso declara origen,

prioridad, precondición, pasos, resultado esperado, nivel de ejecución y evidencia. La matriz completa se reproduce en el Anexo A y su versión editable vive en docs/matriz-pruebas.md.

Tabla 8. Distribución de casos de prueba por historia de usuario y origen.

Historia	Casos	Diseño del equipo	Sugeridos por IA	Aprobados	Pendientes de validación
HU-01 Autenticación	12	8	4	12	0
HU-02 Catálogo	11	7	4	11	0
HU-03 Reserva	18	11	7	18	0
HU-04 Aprobación y rechazo	12	7	5	12	0
HU-05 Devolución y penalización	8	5	3	8	0
Casos IA complementarios	5	0	5	0	5
TOTAL	**66**	**38**	**28**	**61**	**5**

Los casos sugeridos por IA no se presentan como validados. Se listan con su comportamiento esperado y se mantienen en estado pendiente hasta que una persona los revise, los ejecute y firme el resultado.

Tabla 9. Casos de borde sugeridos por IA con validación humana pendiente.

ID	Caso sugerido	Prio.	Nivel	Comportamiento esperado	Validación
IA-01	Reservas contiguas del mismo equipo (10:00–12:00 tras 08:00–10:00)	P1	API + UI	Debe aceptarse: los extremos contiguos no se solapan.	PENDIENTE
IA-02	Solicitud con duración cero (inicio igual a fin)	P1	UI	Debe bloquearse en el cliente antes de enviar la petición.	PENDIENTE
IA-03	Límites exactos del horario (07:00 y 22:00)	P2	API + UI	07:00 y 22:00 se admiten; 06:59 y 22:01 se rechazan con 400.	PENDIENTE
IA-04	Código de estudiante con espacios y minúsculas	P2	API	El backend aplica trim y mayúsculas y permite el acceso legítimo.	PENDIENTE
IA-05	Reintento con token JWT expirado	P1	UI + E2E	El cliente limpia el almacenamiento y redirige a /login sin pantalla en blanco.	PENDIENTE

Definición de las reglas del negocio

Las reglas de negocio se extrajeron del código y de los criterios de aceptación, no de una suposición sobre cómo debería funcionar el sistema. Cada regla indica su implementación real y la prueba que la respalda.

Tabla 10. Reglas de negocio implementadas y su verificación.

ID	Regla de negocio	Implementación	Verificación
RN-01	El laboratorio atiende de 07:00 a 22:00.	Constantes LAB_OPEN y LAB_CLOSE en reservations.service.ts.	reservation-boundaries.extra.spec.ts
RN-02	Una reserva no puede exceder 4 horas continuas.	MAX_RESERVATION_HOURS = 4.	Pruebas de límites del backend
RN-03	La hora de inicio debe ser anterior a la de fin y la fecha no puede ser pasada.	Validación en createReservation.	reservations.spec.ts
RN-04	Un equipo no puede tener dos reservas solapadas.	Consulta de solapamiento dentro de una transacción.	reservations.spec.ts
RN-05	Un estudiante no puede tener dos reservas activas en el mismo horario.	Segunda consulta de solapamiento por usuario.	reservations.spec.ts

ID	Regla de negocio	Implementación	Verificación
RN-06	Solo se reservan equipos disponibles.	Verificación de estado AVAILABLE.	equipment y reservations specs
RN-07	Las cuentas administrativas no registran reservas.	Rechazo explícito por rol.	reservations.spec.ts
RN-08	Tres intentos fallidos bloquean la cuenta.	MAX_FAILED_ATTEMPTS = 3; LOCKOUT_MINUTES = 5.	auth.spec.ts
RN-09	El rechazo exige un motivo de al menos 5 caracteres.	Validación en admin.service.ts.	admin.spec.ts
RN-10	Solo se aprueban o rechazan reservas pendientes; solo se devuelven reservas aprobadas.	Verificación de estado previo.	admin.spec.ts
RN-11	La devolución tardía aplica penalización y cambia el estado del alumno a penalizado.	markReturned con comparación de hora y actualización de usuario.	admin.spec.ts
RN-12	Toda transición relevante se registra en la bitácora.	AuditLog con evento, actor y detalle.	Pruebas de administración

Implementación del proyecto

El proyecto se ejecuta en tres procesos: la API NestJS en el puerto 3000, el frontend de desarrollo en el puerto 5173 y el frontend compilado en el puerto 4173 para las pruebas E2E. La base de pruebas se denomina labtrack_test y se siembra con usuarios y equipos deterministas. Las credenciales de demostración están documentadas en el anexo de reproducción y no se reproducen en el cuerpo del informe.

La compilación exige que TypeScript no emita errores (tsc --noEmit) tanto en backend como en frontend. Esa condición se usó como criterio de cierre de cada refactorización, junto con la ejecución completa de la suite de pruebas.

Pruebas unitarias y de integración

Las pruebas unitarias y de integración se ejecutan con Vitest 3.2.4. En el backend se apoya en Supertest para atacar la API contra una base de pruebas; en el frontend se apoya en React Testing Library y jsdom para verificar componentes y contextos. La suite pasó de 93 a 101 casos aprobados: cuatro nuevos en backend y cuatro en frontend, dedicados a límites de reserva, ventanas contiguas, duración cero y bordes del horario de atención.

Tabla 11**.** Evolución de las pruebas unitarias y de la cobertura de código.

Suite	Antes	Después	Archivos	Cobertura de sentencias	Cobertura de ramas	Estado
Backend (NestJS + Supertest)	49	53	5	95.71 %	85.00 %	100 % aprobadas
Frontend (React + RTL)	44	48	13	90.26 %	81.69 %	100 % aprobadas
TOTAL	**93**	**101**	**18**	**91.50 % global**	**82.50 % global**	**Código de salida 0**

Además de las pruebas verdes, la suite incluye casos con it.fails que registran la falla esperada de funciones defectuosas simuladas en mocks locales. Esa técnica

demuestra que las pruebas tienen poder de detección: una prueba que nunca puede fallar no aporta información sobre la calidad del sistema.



Figura 7**.** Salida de Vitest del backend después de incorporar los casos de límites (53 pruebas aprobadas).

DESPUES - Frontend (React + Vitest): suite completa con casos límite nuevos

Comando: cd frontend && npx vitest run | Ejecutado: 2026-11-09 18:13 | Commit: 0a63a0a | Salida real sin editar

```
 RUN  v3.2.7 C:/Users/1luis/Documents/Calidad y Pruebas de software/T3/frontend

node.exe : stderr | src/test/ProtectedRoute.test.tsx > ProtectedRoute · control de acceso > redirige al login cuando
no hay sesión
En línea: 1 Carácter: 1
+ & "C:\Program Files\nodejs/node.exe" "C:\Program Files\nodejs/node_mo ...
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : NotSpecified: (stderr | src/te...o no hay sesión:String) [], RemoteException
    + FullyQualifiedErrorId : NativeCommandError

⚠ React Router Future Flag Warning: React Router will begin wrapping state updates in `React.startTransition` in v7.
You can use the `v7_startTransition` future flag to opt-in early. For more information, see
https://reactrouter.com/v6/upgrading/future#v7_starttransition.
⚠ React Router Future Flag Warning: Relative route resolution within Splat routes is changing in v7. You can use the
`v7_relativeSplatPath` future flag to opt-in early. For more information, see
https://reactrouter.com/v6/upgrading/future#v7_relativesplatpath.

stderr | src/test/MyReservationsPage.test.tsx > MyReservationsPage · seguimiento > muestra el motivo de rechazo al
estudiante
⚠ React Router Future Flag Warning: React Router will begin wrapping state updates in `React.startTransition` in v7.
You can use the `v7_startTransition` future flag to opt-in early. For more information, see
https://reactrouter.com/v6/upgrading/future#v7_starttransition.
⚠ React Router Future Flag Warning: Relative route resolution within Splat routes is changing in v7. You can use the
`v7_relativeSplatPath` future flag to opt-in early. For more information, see
https://reactrouter.com/v6/upgrading/future#v7_relativesplatpath.

 ✓ src/test/ProtectedRoute.test.tsx (4 tests) 112ms
stderr | src/test/AppLayout.test.tsx > AppLayout · navegación > muestra la navegación del estudiante y su identidad
⚠ React Router Future Flag Warning: React Router will begin wrapping state updates in `React.startTransition` in v7.
You can use the `v7_startTransition` future flag to opt-in early. For more information, see
https://reactrouter.com/v6/upgrading/future#v7_starttransition.
⚠ React Router Future Flag Warning: Relative route resolution within Splat routes is changing in v7. You can use the
`v7_relativeSplatPath` future flag to opt-in early. For more information, see
https://reactrouter.com/v6/upgrading/future#v7_relativesplatpath.

stderr | src/test/NewReservationPage.test.tsx > NewReservationPage · HU-03 > preselecciona el equipo enviado desde el
catálogo
⚠ React Router Future Flag Warning: React Router will begin wrapping state updates in `React.startTransition` in v7.
You can use the `v7_startTransition` future flag to opt-in early. For more information, see
https://reactrouter.com/v6/upgrading/future#v7_starttransition.
⚠ React Router Future Flag Warning: Relative route resolution within Splat routes is changing in v7. You can use the
`v7_relativeSplatPath` future flag to opt-in early. For more information, see
https://reactrouter.com/v6/upgrading/future#v7_relativesplatpath.

stderr | src/test/AdminPendingPage.test.tsx > AdminPendingPage · HU-04 > lista las solicitudes con los datos del
estudiante y del equipo
⚠ React Router Future Flag Warning: React Router will begin wrapping state updates in `React.startTransition` in v7.
You can use the `v7_startTransition` future flag to opt-in early. For more information, see
https://reactrouter.com/v6/upgrading/future#v7_starttransition.
⚠ React Router Future Flag Warning: Relative route resolution within Splat routes is changing in v7. You can use the
`v7_relativeSplatPath` future flag to opt-in early. For more information, see
https://reactrouter.com/v6/upgrading/future#v7_relativesplatpath.

stderr | src/test/LoginPage.test.tsx > LoginPage · HU-01 > muestra el formulario y completa las credenciales de
demostración
⚠ React Router Future Flag Warning: React Router will begin wrapping state updates in `React.startTransition` in v7.
You can use the `v7_startTransition` future flag to opt-in early. For more information, see
https://reactrouter.com/v6/upgrading/future#v7_starttransition.
⚠ React Router Future Flag Warning: Relative route resolution within Splat routes is changing in v7. You can use the
`v7_relativeSplatPath` future flag to opt-in early. For more information, see
https://reactrouter.com/v6/upgrading/future#v7_relativesplatpath.

stderr | src/test/App.test.tsx > App · enrutamiento y sesión > muestra el login cuando no hay sesión
⚠ React Router Future Flag Warning: React Router will begin wrapping state updates in `React.startTransition` in v7.
You can use the `v7_startTransition` future flag to opt-in early. For more information, see
https://reactrouter.com/v6/upgrading/future#v7_starttransition.
⚠ React Router Future Flag Warning: Relative route resolution within Splat routes is changing in v7. You can use the
`v7_relativeSplatPath` future flag to opt-in early. For more information, see
https://reactrouter.com/v6/upgrading/future#v7_relativesplatpath.

stderr | src/test/AdminHistoryPage.test.tsx > AdminHistoryPage · trazabilidad > lista las reservas con su estado y
revisor
⚠ React Router Future Flag Warning: React Router will begin wrapping state updates in `React.startTransition` in v7.
You can use the `v7_startTransition` future flag to opt-in early. For more information, see
https://reactrouter.com/v6/upgrading/future#v7_starttransition.
⚠ React Router Future Flag Warning: Relative route resolution within Splat routes is changing in v7. You can use the
`v7_relativeSplatPath` future flag to opt-in early. For more information, see
https://reactrouter.com/v6/upgrading/future#v7_relativesplatpath.

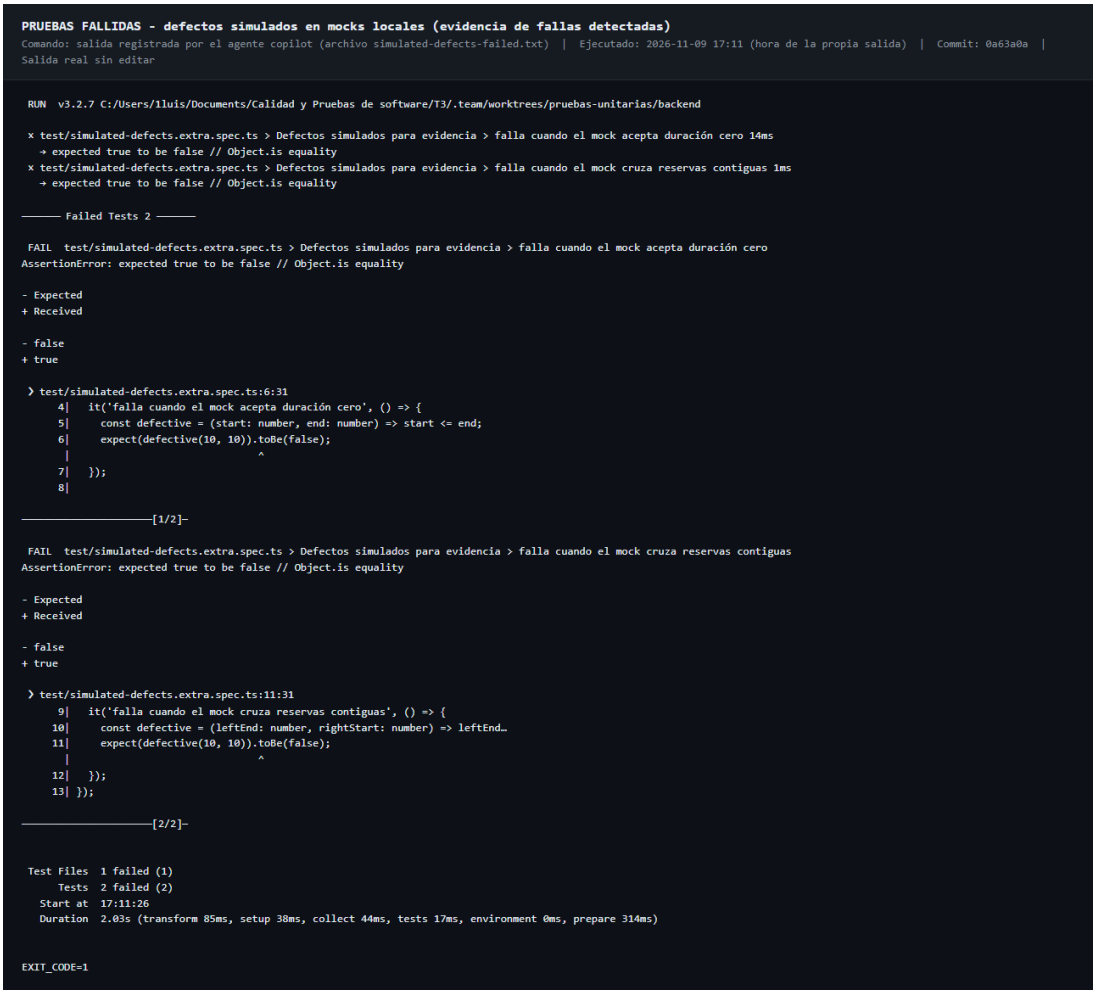
stderr | src/test/AdminReturnsPage.test.tsx > AdminReturnsPage · HU-05 > marca visualmente un préstamo vencido
⚠ React Router Future Flag Warning: React Router will begin wrapping state updates in `React.startTransition` in v7.
You can use the `v7_startTransition` future flag to opt-in early. For more information, see
https://reactrouter.com/v6/upgrading/future#v7_starttransition.
⚠ React Router Future Flag Warning: Relative route resolution within Splat routes is changing in v7. You can use the
`v7_relativeSplatPath` future flag to opt-in early. For more information, see
https://reactrouter.com/v6/upgrading/future#v7_relativesplatpath.

stderr | src/test/CatalogPage.test.tsx > CatalogPage · HU-02 > muestra las tarjetas con nombre, código, categoría y
estado
⚠ React Router Future Flag Warning: React Router will begin wrapping state updates in `React.startTransition` in v7.
You can use the `v7_startTransition` future flag to opt-in early. For more information, see
https://reactrouter.com/v6/upgrading/future#v7_starttransition.
⚠ React Router Future Flag Warning: Relative route resolution within Splat routes is changing in v7. You can use the
`v7_relativeSplatPath` future flag to opt-in early. For more information, see
https://reactrouter.com/v6/upgrading/future#v7_relativesplatpath.

stderr | src/test/MiscComponents.test.tsx > Componentes compartidos > la página 404 informa el error y ofrece volver
al inicio
⚠ React Router Future Flag Warning: React Router will begin wrapping state updates in `React.startTransition` in v7.
You can use the `v7_startTransition` future flag to opt-in early. For more information, see
https://reactrouter.com/v6/upgrading/future#v7_starttransition.
⚠ React Router Future Flag Warning: Relative route resolution within Splat routes is changing in v7. You can use the
`v7_relativeSplatPath` future flag to opt-in early. For more information, see
https://reactrouter.com/v6/upgrading/future#v7_relativesplatpath.

 ✓ src/test/MiscComponents.test.tsx (4 tests) 280ms
 ✓ src/test/AppLayout.test.tsx (4 tests) 640ms
 ✓ src/test/MyReservationsPage.test.tsx (4 tests) 544ms
 ✓ src/test/App.test.tsx (3 tests) 453ms
 ✓ src/test/AdminHistoryPage.test.tsx (3 tests) 562ms
 ✓ src/test/AdminReturnsPage.test.tsx (2 tests) 547ms
   ✓ AdminReturnsPage · HU-05 > advierte la penalización y registra la devolución tardía  409ms
 ✓ src/test/NewReservationPage.test.tsx (4 tests) 698ms
 ✓ src/test/AdminPendingPage.test.tsx (3 tests) 1062ms
   ✓ AdminPendingPage · HU-04 > exige un motivo antes de rechazar una solicitud  842ms
 ✓ src/test/CatalogPage.test.tsx (5 tests) 921ms
   ✓ CatalogPage · HU-02 > aplica el filtro exacto por categoría  312ms
 ✓ src/test/LoginPage.test.tsx (4 tests) 1840ms
   ✓ LoginPage · HU-01 > muestra el mensaje "Credenciales inválidas" ante un rechazo del API  599ms
   ✓ LoginPage · HU-01 > muestra la alerta de bloqueo temporal  499ms
   ✓ LoginPage · HU-01 > guarda la sesión cuando las credenciales son válidas  471ms
 ✓ src/test/reservation-boundaries.extra.test.ts (4 tests) 10ms
```

Figura 8. Salida de Vitest del frontend después de incorporar los casos de límites (48 pruebas aprobadas).



```
PRUEBAS FALLIDAS - defectos simulados en mocks locales (evidencia de fallas detectadas)
Comando: salida registrada por el agente copilot (archivo simulated-defects-failed.txt)  |  Ejecutado: 2026-11-09 17:11 (hora de la propia salida)  |  Commit: 0a63a0a  |
Salida real sin editar

 RUN  v3.2.7 C:/Users/1luis/Documents/Calidad y Pruebas de software/T3/.team/worktrees/pruebas-unitarias/backend

 × test/simulated-defects.extra.spec.ts > Defectos simulados para evidencia > falla cuando el mock acepta duración cero 14ms
   → expected true to be false // Object.is equality
 × test/simulated-defects.extra.spec.ts > Defectos simulados para evidencia > falla cuando el mock cruza reservas contiguas 1ms
   → expected true to be false // Object.is equality

⎯⎯⎯⎯⎯⎯ Failed Tests 2 ⎯⎯⎯⎯⎯⎯

 FAIL  test/simulated-defects.extra.spec.ts > Defectos simulados para evidencia > falla cuando el mock acepta duración cero
AssertionError: expected true to be false // Object.is equality

- Expected
+ Received

- false
+ true

 ❯ test/simulated-defects.extra.spec.ts:6:31
      4|   it('falla cuando el mock acepta duración cero', () => {
      5|     const defective = (start: number, end: number) => start <= end;
      6|     expect(defective(10, 10)).toBe(false);
       |                               ^
      7|   });
      8|

⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯[1/2]⎯

 FAIL  test/simulated-defects.extra.spec.ts > Defectos simulados para evidencia > falla cuando el mock cruza reservas contiguas
AssertionError: expected true to be false // Object.is equality

- Expected
+ Received

- false
+ true

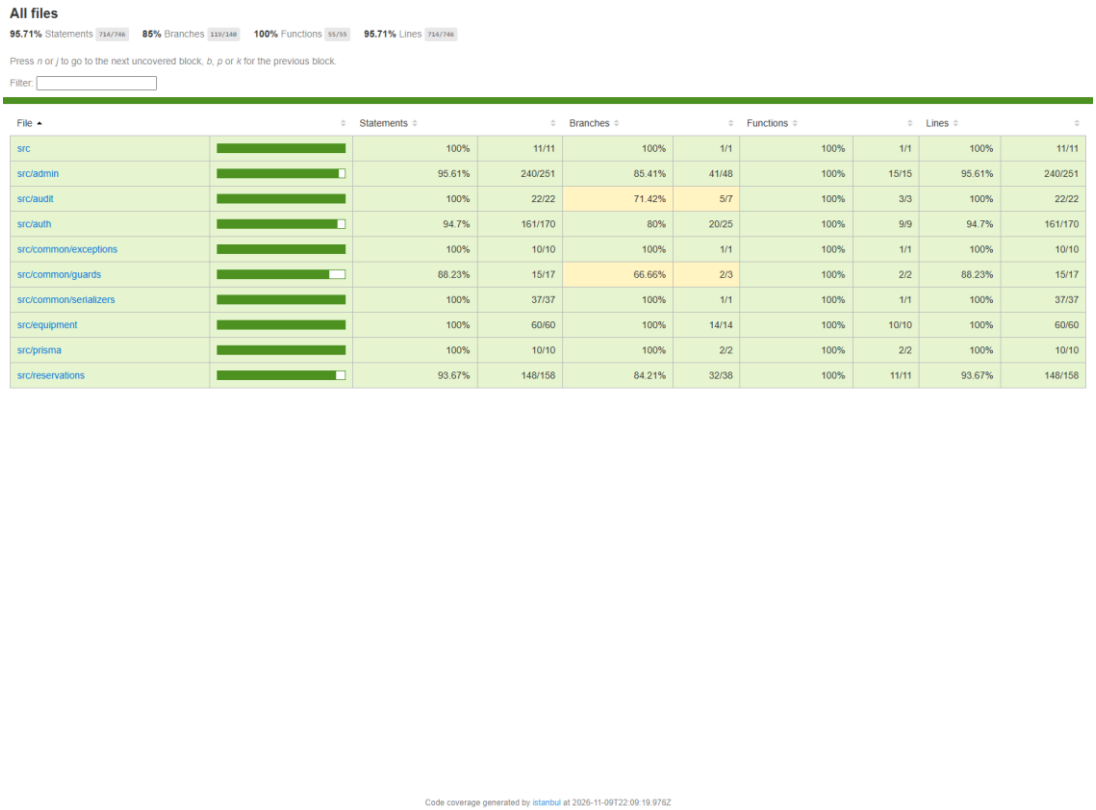
 ❯ test/simulated-defects.extra.spec.ts:11:31
      9|   it('falla cuando el mock cruza reservas contiguas', () => {
     10|     const defective = (leftEnd: number, rightStart: number) => leftEnd…
     11|     expect(defective(10, 10)).toBe(false);
       |                               ^
     12|   });
     13| });

⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯[2/2]⎯


 Test Files  1 failed (1)
      Tests  2 failed (2)
   Start at  17:11:26
   Duration  2.03s (transform 85ms, setup 38ms, collect 44ms, tests 17ms, environment 0ms, prepare 314ms)


EXIT_CODE=1
```

Figura 9. Ejecución contra mocks defectuosos: la suite detecta el fallo y termina con código de salida 1.



All files

95.71% Statements 714/746 85% Branches 119/140 100% Functions 55/55 95.71% Lines 714/746

Press n or j to go to the next uncovered block, b, p or k for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
src	100%	11/11	100%	1/1	100%	1/1	100%	11/11
src/admin	95.61%	240/251	85.41%	41/48	100%	15/15	95.61%	240/251
src/audit	100%	22/22	71.42%	5/7	100%	3/3	100%	22/22
src/auth	94.7%	161/170	80%	20/25	100%	9/9	94.7%	161/170
src/common/exceptions	100%	10/10	100%	1/1	100%	1/1	100%	10/10
src/common/guards	88.23%	15/17	66.66%	2/3	100%	2/2	88.23%	15/17
src/common/serializers	100%	37/37	100%	1/1	100%	1/1	100%	37/37
src/equipment	100%	60/60	100%	14/14	100%	10/10	100%	60/60
src/prisma	100%	10/10	100%	2/2	100%	2/2	100%	10/10
src/reservations	93.67%	148/158	84.21%	32/38	100%	11/11	93.67%	148/158

Code coverage generated by istanbul at 2026-11-09T22:09:19.976Z

Figura 10. Reporte HTML de cobertura del backend generado con Istanbul/V8.

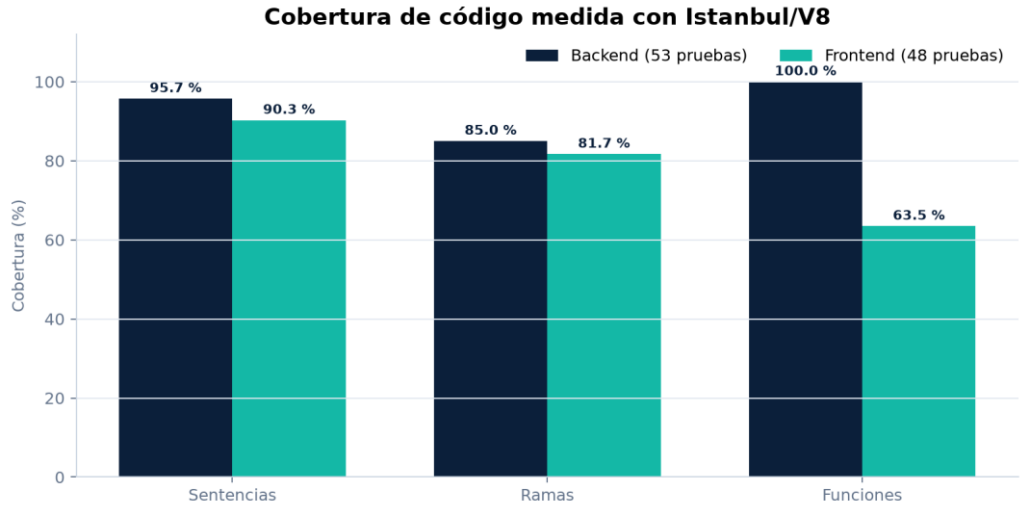


Figura 11. Comparativa de cobertura entre backend y frontend medida con Istanbul/V8.

Pruebas funcionales con Selenium

La suite de extremo a extremo se implementó con Selenium WebDriver 4.27 sobre Google Chrome headless y consta de siete pruebas en frontend/e2e/critical-flows.e2e.test.ts. El entorno de ejecución fue el backend en localhost:3000 con la base labtrack_test y el frontend compilado en localhost:4173. Los resultados se documentan en JUnit XML y en capturas del navegador.

Tabla 12**.** Corridas reales de la suite Selenium y resultado obtenido.

Corrida	Selectores utilizados	Resultado	Detalle
1	[role="status"] y [role="dialog"] sobre el código ya refactorizado	4 aprobadas / 3 fallidas	Fallan HU-03, HU-04 y HU-05: el aviso y el diálogo ya no exponen el atributo role explícito.
2	Ajuste intermedio de los localizadores	Uso interno de diagnóstico	Se identificó que el comportamiento del sistema era correcto y el fallo estaba en la prueba.
3 (final)	output, [role="status"] y dialog, [role="dialog"]	7 aprobadas / 0 fallidas	JUnit XML sin fallos y capturas 01 a 08 de la corrida final.

Regresión detectada: la refactorización S6819 rompió los selectores E2E

La corrección de la regla S6819 sustituyó por <output> y por <dialog>. Ambos elementos ya poseen el rol de forma implícita, de modo que la accesibilidad mejoró y el comportamiento funcional se mantuvo. Sin embargo, los selectores CSS [role="status"] y [role="dialog"] solo encuentran el atributo escrito, no el rol implícito; por eso las pruebas dejaron de localizar el aviso y el cuadro de confirmación. El sistema creaba la reserva correctamente: lo que fallaba era la prueba. La corrección se aplicó solo en frontend/e2e/helpers/e2e.ts y frontend/e2e/critical-flows.e2e.test.ts, sin tocar el código de producción. La lección se incorporó al backlog: toda refactorización de accesibilidad obliga a reejecutar la suite E2E.

Este hallazgo es el resultado más valioso de la automatización. Una corrida de 7 de 7 desde el primer intento habría ocultado una dependencia frágil entre las pruebas y el marcado. La regresión quedó documentada con su causa, su corrección y su evidencia, y alimenta una regla de proceso para el futuro.

Tabla 13**.** Flujos críticos automatizados con Selenium y evidencia asociada.

Flujo automatizado	HU	Comportamiento validado	Captura
Rechazo de credenciales inválidas	HU-01	Alerta con el texto exacto «Credenciales inválidas.»	01-login-credenciales-invalidas.png
Bloqueo tras tres intentos fallidos	HU-01	Alerta de bloqueo temporal y respuesta 423	02-login-cuenta-bloqueada.png
Inicio de sesión y catálogo	HU-01, HU-02	Ingreso exitoso y renderizado de tarjetas	03-catalogo-estudiante.png
Filtro por categoría Electrónica	HU-02	Filtrado reactivo del catálogo	04-filtro-electronica.png
Registro de reserva	HU-03	Reserva pendiente y redirección a Mis Reservas	05-reserva-registrada.png
Aprobación administrativa	HU-04	Cambio a APPROVED y equipo a BORROWED	06-solicitud-aprobada.png
Devolución tardía con penalización	HU-05	Detección del retraso y sanción al alumno	07-devolucion-penalizada.png

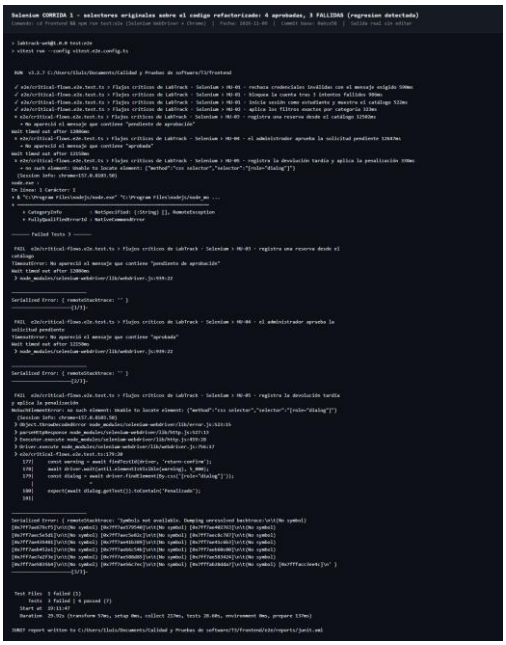


Figura 12**.** Consola de la primera corrida: 4 pruebas aprobadas y 3 fallidas por la regresión de selectores.

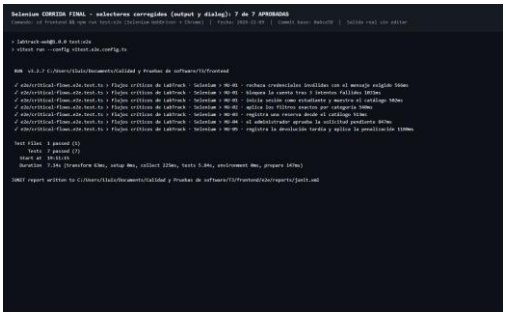


Figura 13**.** Consola de la corrida final: 7 pruebas aprobadas y 0 fallidas.

Herramientas de gestión de pruebas. Testomat.io, TestLink y Katalon Studio no se ejecutaron: requieren cuenta o servidor y no formaban parte del entorno disponible. Para no fabricar evidencias, se entrega la plantilla de importación y el archivo docs/evidencias-pruebas/testomatio-import.csv, listos para cargar los casos cuando la cátedra habilite el acceso. Esta limitación se declara también en la conclusión y en las recomendaciones.

Documentación de la refactorización

La refactorización no fue una colección de parches: fue una estrategia para eliminar deuda técnica sin alterar el comportamiento observable. Cada cambio se acompañó de la suite completa de pruebas y de un nuevo análisis estático. La tabla siguiente resume las diez reglas que concentraban las 44 incidencias iniciales.

Tabla 14. Hallazgos de SonarQube que concentraban la deuda técnica inicial.

Regla	Hallazgo	Tipo y severidad	Cantidad	Archivos principales	Corrección aplicada
S3358	Ternarios anidados	Code smell · Major	8	admin.service.ts y páginas React	Sustitución por if/else y variables auxiliares
S6759	Props sin Readonly	Code smell · Major	14	Componentes, contextos y páginas	Readonly<Props> en componentes funcionales
S7718	Parámetro catch fuera de convención	Code smell · Major	11	Once archivos de páginas	Renombrado a error_
S6819	Rol ARIA en lugar de HTML nativo	Code smell · Major	5	ui.tsx, ToastContext, CatalogPage	Migración a dialog y output
S2068	Credencial embebida	Vulnerabilidad · Blocker	1	LoginPage.tsx	Variables de entorno VITE_DEMO_*
S2245	PRNG inseguro	Vulnerabilidad · Critical	1	ToastContext.tsx	Secuencia monotónica en memoria
S1082	Clic sin soporte de teclado	Bug · Minor	1	ui.tsx	Botón de cierre accesible y dialog nativo
S6847	Evento interactivo en elemento no interactivo	Code smell · Major	1	ui.tsx	Aislamiento modal semántico con dialog
S9379	autoFocus en campo de entrada	Code smell · Minor	1	LoginPage.tsx	Eliminación del atributo
S3776	Complejidad cognitiva alta	Code smell · Critical	1	AdminHistoryPage.tsx	Extracción de bloques de renderizado

Se reproducen tres ejemplos representativos. El primero corresponde al bug de accesibilidad: un contenedor con manejador de clic y sin interacción de teclado. El segundo ilustra la vulnerabilidad más grave: una contraseña en texto plano. El tercero muestra la corrección que, sin proponérselo, rompió los selectores de la suite E2E.

```
// ANTES (S1082 + S6847)
<div onClick={onClose} role="presentation">
  <div role="dialog" aria-modal="true" aria-label={title}
       onClick={(event) => event.stopPropagation()}>
    {/* contenido */}
  </div>
</div>

// DESPUÉS (HTML nativo y cierre accesible)
<div className="fixed inset-0 z-[90] flex items-end justify-center p-4">
  <button type="button" aria-label="Cerrar modal" onClick={onClose}
          className="absolute inset-0 bg-navy-950/50 backdrop-blur-sm" />
  <dialog open aria-modal="true" aria-label={title}
          className="relative m-0 w-full max-w-lg rounded-2xl border-0 bg-white p-0 shadow-
lift">
    {/* contenido */}
  </dialog>
</div>
```

```
// ANTES (S2068): credencial en texto plano
password: 'Estudiante2026!'

// DESPUÉS: variable de entorno
password: import.meta.env.VITE_DEMO_STUDENT_PASSWORD ?? ''
```

```
// ANTES (S6819): rol ARIA sobre un div
<div role="status" aria-live="polite">{toast.message}</div>
<div role="dialog" aria-modal="true" aria-label={title}>

// DESPUÉS: elementos nativos con el mismo rol implícito
<output aria-live="polite">{toast.message}</output>
```

```
<dialog open aria-modal="true" aria-label={title}>

// Consecuencia en las pruebas: [role="status"] deja de encontrarlo.
// Corrección en el test, no en producción:
const toast = await driver.findElement(By.css('output, [role="status"]'));
const modal = await driver.findElement(By.css('dialog, [role="dialog"]'));
```

El backlog de refactorización registra catorce ítems técnicos y nueve brechas funcionales. Los dos últimos ítems son los más relevantes para este informe: la regresión de Selenium provocada por S6819 y el defecto de contraste detectado por axe-core, que permanece abierto.

Tabla 15**.** Backlog de refactorización con estado y evidencia de cierre.

ID	Hallazgo	Prioridad	Solución	Estado	Evidencia
RF-01	Ternarios anidados (S3358)	Alta	if/else explícitos	Completado	8 incidencias cerradas
RF-02	Modal inaccesible (S1082, S6819, S6847)	Alta	dialog y botón de cierre	Completado	Bug cerrado; confiabilidad A
RF-03	PRNG inseguro (S2245)	Alta	Secuencia monotónica	Completado	Vulnerabilidad cerrada
RF-04	Credencial embebida (S2068)	Alta	Variables VITE_DEMO_*	Completado	Seguridad A
RF-05	Props sin Readonly (S6759)	Media	Readonly<Props>	Completado	14 incidencias cerradas
RF-06	Catch fuera de convención (S7718)	Media	Renombrado a error_	Completado	11 incidencias cerradas
RF-07	autoFocus (S9379)	Media	Eliminación del atributo	Completado	Incidencia cerrada
RF-08	Complejidad cognitiva (S3776)	Media	Extracción de bloques	Completado	Complejidad < 15
RF-09	Cobertura no reportada al scanner	Media	Rutas LCOV en la configuración	Completado	Reportes V8 archivados
RF-10	Casos límite sin pruebas	Media	Suites de límites	Completado	93 → 101 pruebas
RF-11	Credenciales demo en el seed	Baja	Aislamiento del entorno	Aceptado	Riesgo documentado
RF-12	E2E solo en Chrome	Baja	Parametrizar Firefox y Edge	Planificado	Sprint siguiente
RF-13	Regresión de selectores E2E por S6819	Alta	Selectores que aceptan rol implícito en helpers y flujos críticos	Resuelto	Corrida final 7/7 y JUnit XML
RF-14	Contraste de color insuficiente (axe-core)	Alta	Ajustar paleta y revalidar con axe	Pendiente	1 elemento en login y 17 en catálogo

La regresión RF-13 se resolvió modificando únicamente los archivos de prueba (frontend/e2e/helpers/e2e.ts y critical-flows.e2e.test.ts). El código de producción no se alteró, precisamente porque su comportamiento era correcto. El defecto RF-14 no se corrigió: se documenta con su impacto, sus elementos afectados y la acción requerida.

Análisis estático: SonarQube antes y después

El análisis se ejecutó con SonarQube Community Build 26.10.0.133078 sobre PostgreSQL 18, con SonarScanner CLI 8.3.1.16491 y OpenJDK 21. Se aplicó el perfil Sonar way para TypeScript y JavaScript sobre backend/src y frontend/src, excluyendo node_modules, dist, coverage, declaraciones y pruebas. El estado ANTES se reproduce en una copia de trabajo porque el repositorio no conserva una versión previa; el estado DESPUÉS se analiza sobre el código actual.

Transparencia metodológica sobre el estado ANTES y la cobertura

1. El código ANTES es REPRODUCIDO, no histórico. El repositorio tiene un único commit; para comparar en el mismo servidor y con los mismos parámetros se reinyectaron los 44 defectos documentados en una copia de trabajo. Las capturas del ANTES provienen de esa ejecución real, no de un panel reconstruido.
2. La cobertura reportada por SonarQube fue 0.0 % porque no se inyectaron las rutas LCOV en esa ejecución puntual. Es un defecto de configuración, registrado como RF-09. La cobertura real, medida con Istanbul/V8, es 95.71 % en backend y 90.26 % en frontend.
3. La puerta de calidad se aprobó en ambos estados porque evalúa código nuevo. Aprobar la puerta no equivale a tener cero deuda: el valor está en la comparación de incidencias totales.

Tabla 16**.** Comparación de métricas de SonarQube antes y después de la refactorización.

Métrica	Antes	Después	Variación	Lectura
Bugs	1	0	−1 (−100 %)	Se elimina la falla de accesibilidad con teclado
Vulnerabilidades	2	0	−2 (−100 %)	Sin credenciales expuestas ni PRNG predecible
Code smells	41	0	−41 (−100 %)	Código más legible y desacoplado
Confiabilidad	B	A	B → A	Sin fallos potenciales detectados
Seguridad	C	A	C → A	Cierre de S2068 y S2245
Mantenibilidad	A	A	Se mantiene	Deuda técnica de 1 h 37 min a 0
Líneas de código (NCLOC)	4 145	4 120	−25	Eliminación de redundancias
Duplicación	0.7 %	0.7 %	Sin cambio	Muy por debajo del umbral del 3 %
Quality Gate	Aprobada	Aprobada	Sin cambio	Evalúa código nuevo en ambos casos

El total de 44 incidencias coincide con la línea base histórica registrada en docs/sonar-baseline.json (2 bugs, 2 vulnerabilidades y 40 smells). La distribución por tipo difiere levemente (1 bug y 41 smells) porque la versión 26.10 del motor reclasificó uno de los elementos interactivos y computó la regla de complejidad cognitiva S3776. La diferencia se explica por la evolución del motor, no por una manipulación de los datos.

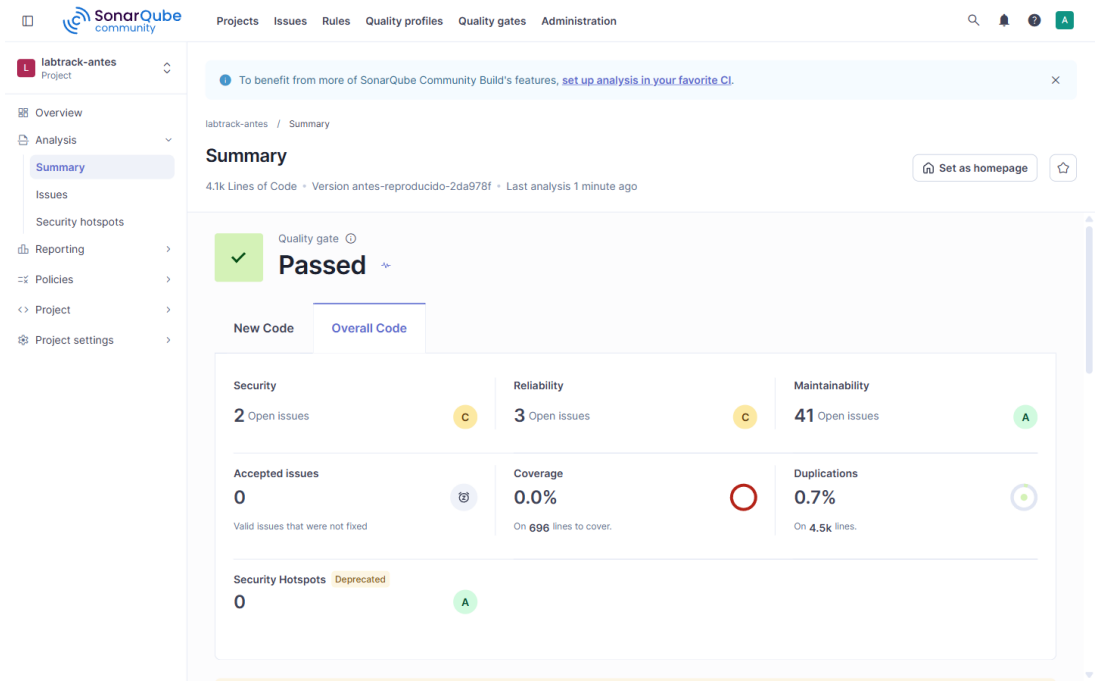


Figura 14. SonarQube, estado ANTES reproducido: 44 incidencias, confiabilidad B y seguridad C.

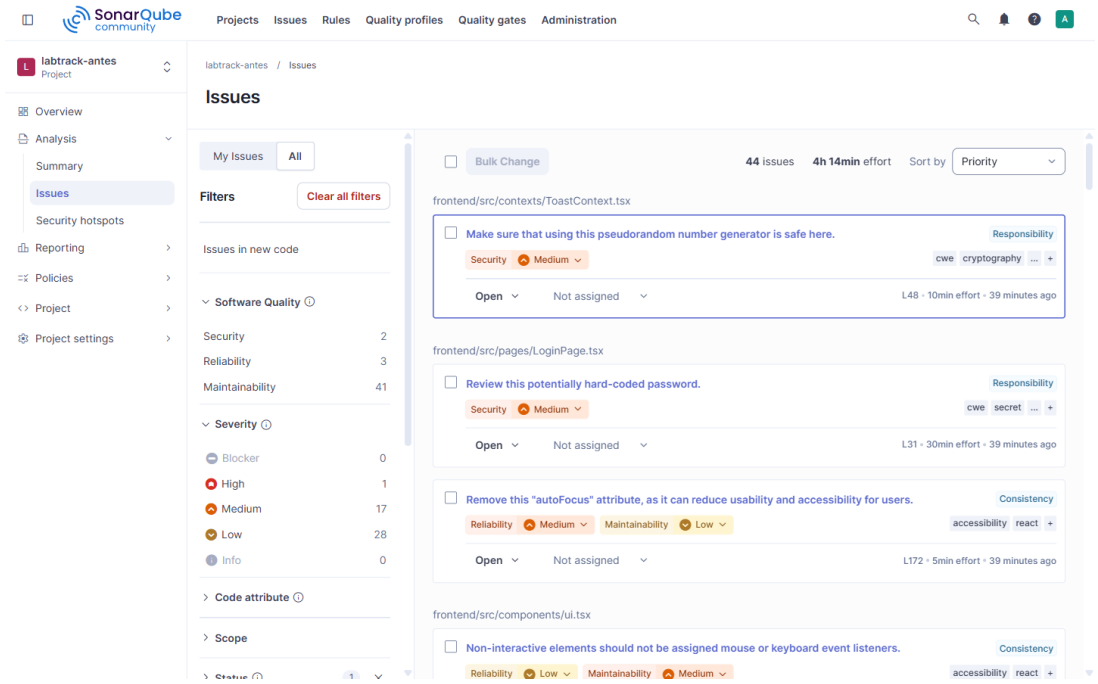


Figura 15. Listado completo de las 44 incidencias del estado ANTES con regla y archivo.

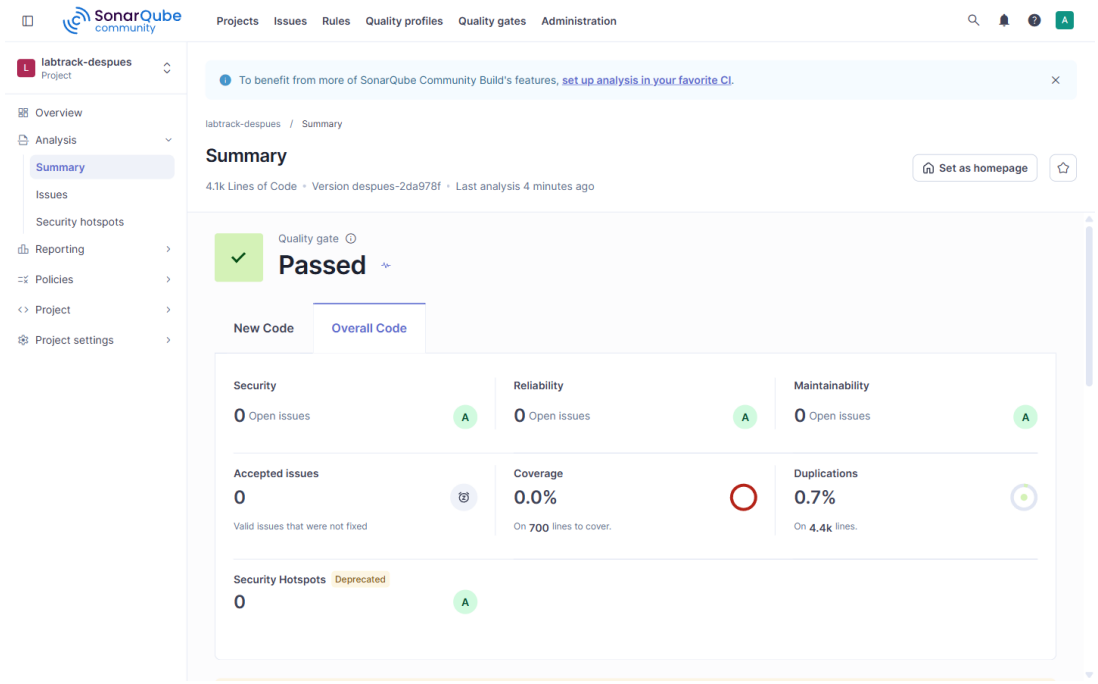


Figura 16. SonarQube, estado DESPUÉS: 0 incidencias y calificaciones A/A/A.

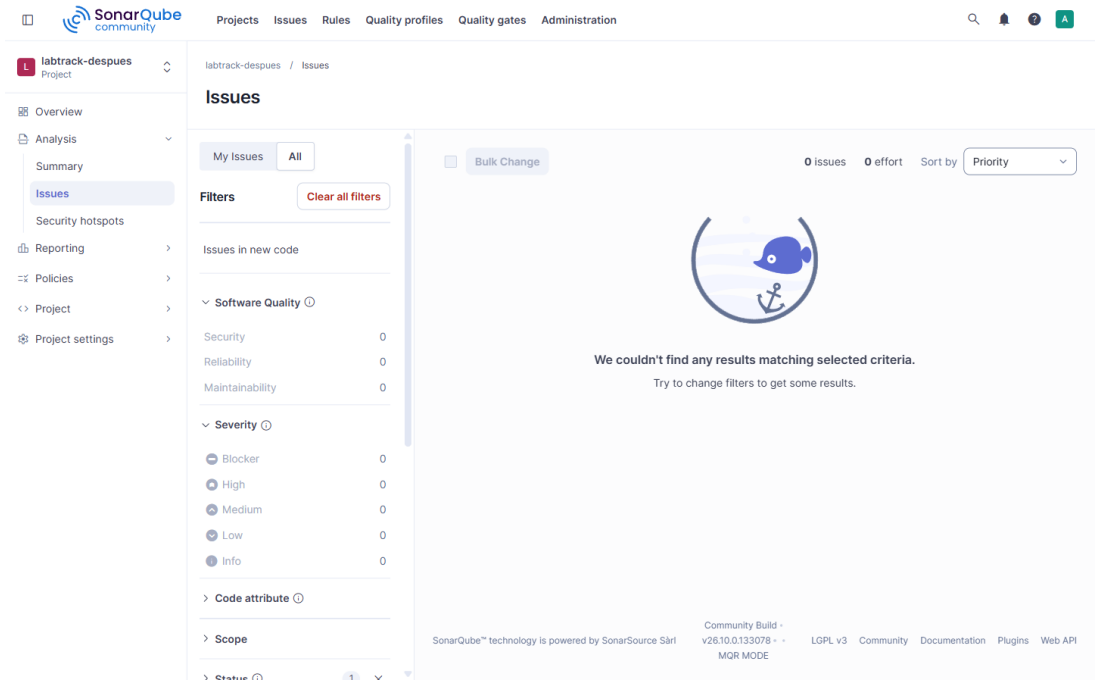


Figura 17. Bandeja de incidencias vacía después de la refactorización.

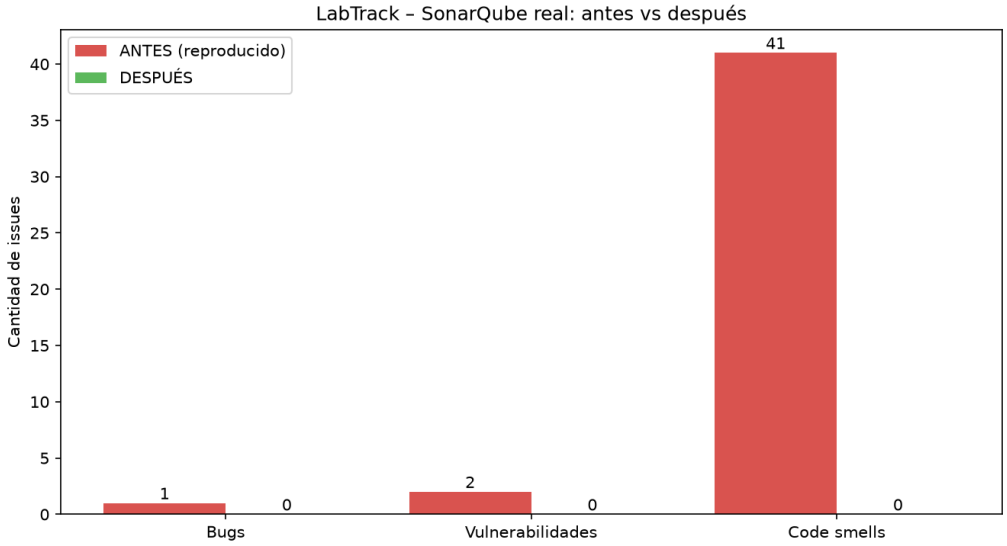


Figura 18. Comparativo real de incidencias de SonarQube antes y después.

Tabla 17. Matriz consolidada de los pilares de calidad de SonarQube.

Pilar	Antes	Después	Condición exigida	Resultado
Confiabilidad	B (1 bug)	A (0 bugs)	A sin bugs abiertos	Conforme
Seguridad	C (2 vulnerabilidades)	A (0 vulnerabilidades)	A sin vulnerabilidades	Conforme
Mantenibilidad	A (41 smells)	A (0 smells)	A con deuda < 5 %	Conforme
Hotspots de seguridad	0	0	100 % revisados	Conforme
Duplicación	0.7 %	0.7 %	< 3.0 %	Conforme
Quality Gate	Aprobada	Aprobada	Sonar way	Conforme

Pruebas de carga y estrés con Apache JMeter

Se ejecutaron dos escenarios con Apache JMeter 5.6.3. La prueba de carga simuló 50 usuarios concurrentes durante 60 segundos con rampa de 10 segundos; la prueba de estrés elevó la concurrencia a 150 usuarios durante 90 segundos con rampa de 15 segundos. El guion recorrió inicio de sesión con JWT, catálogo completo, filtro por categoría, estadísticas y consultas de reservas del estudiante.

El percentil 95 se adoptó como métrica principal porque el promedio oculta la latencia de cola. Un sistema puede promediar 30 ms y, al mismo tiempo, castigar a un grupo pequeño de usuarios con esperas de varios segundos. El p95 responde a la pregunta relevante para el usuario: ¿cuánto espera el 95 % de las peticiones?

Tabla 18. Resumen global de la prueba de carga con 50 usuarios concurrentes.

Métrica	Resultado	Umbral	Estado
Muestras procesadas	79 424	Volumen informativo	Informativo
Tasa de error	0.00 %	≤ 1.0 %	Cumple
Throughput	1 334.07 req/s	Alta capacidad	Excelente
Promedio	34.4 ms	Informativo	Informativo
Mediana (p50)	34.0 ms	≤ 3000 ms	Cumple
p90	42.0 ms	≤ 3000 ms	Cumple

Métrica	Resultado	Umbral	Estado
p95	48.0 ms	≤ 3000 ms	Cumple
p99	89.0 ms	Informativo	Informativo
Máximo	230.0 ms	≤ 3000 ms	Cumple
Desviación estándar	11.8 ms	Informativo	Alta estabilidad

Tabla 19**.** Detalle por transacción de la prueba de carga.

Transacción	Muestras	Promedio	p95	p99	Throughput	Error
GET /api/equipment	19 865	34.5 ms	49.0 ms	89.0 ms	333.67 req/s	0.00 %
GET /api/equipment/stats	19 836	34.4 ms	48.0 ms	88.0 ms	333.18 req/s	0.00 %
GET /api/equipment?category=Cómputo	19 848	34.3 ms	48.0 ms	88.5 ms	333.38 req/s	0.00 %
GET /api/reservations/me	19 825	34.3 ms	48.0 ms	88.0 ms	333.00 req/s	0.00 %
POST /api/auth/login	50	113.5 ms	160.8 ms	204.5 ms	0.84 req/s	0.00 %
TOTAL	**79 424**	**34.4 ms**	**48.0 ms**	**89.0 ms**	**1 334.07 req/s**	**0.00 %**

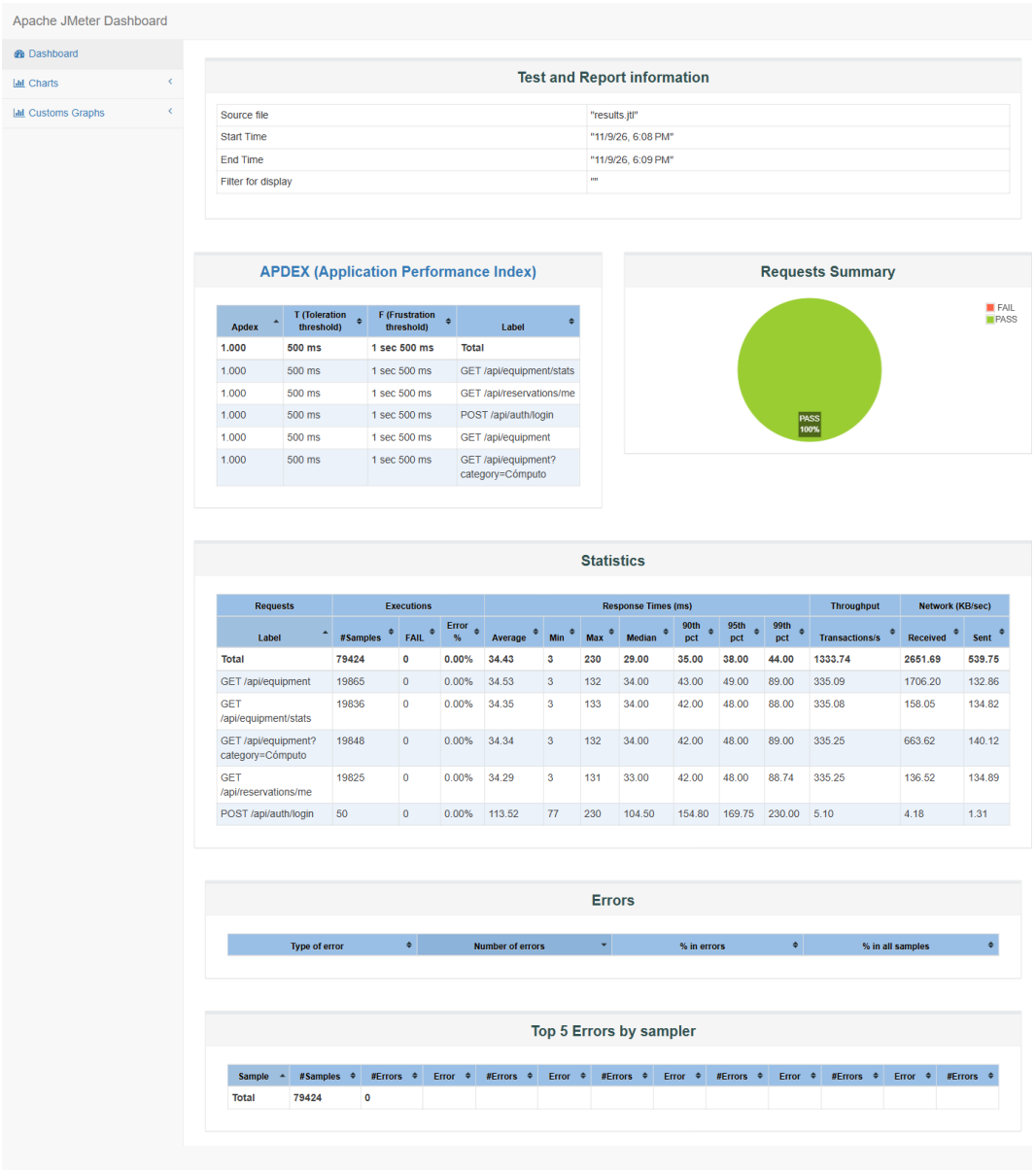


Figura 19**.** Dashboard de JMeter de la prueba de carga: 0.00 % de error sobre 79 424 peticiones.

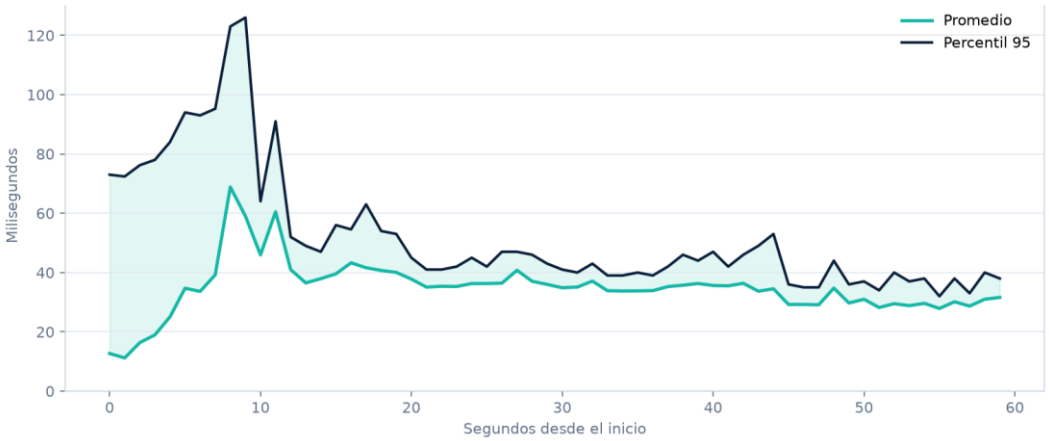


Figura 20. Tiempo de respuesta (promedio y percentil 95) durante la prueba de carga con 50 usuarios.

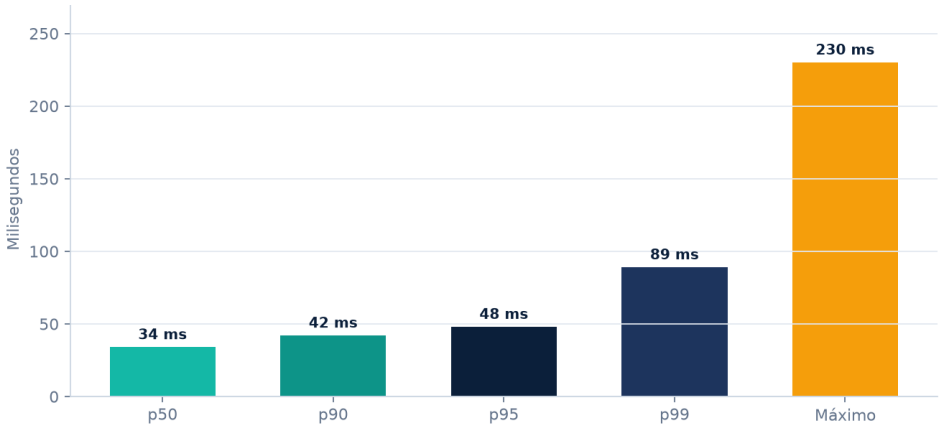


Figura 21. Distribución de percentiles de la prueba de carga con 50 usuarios.

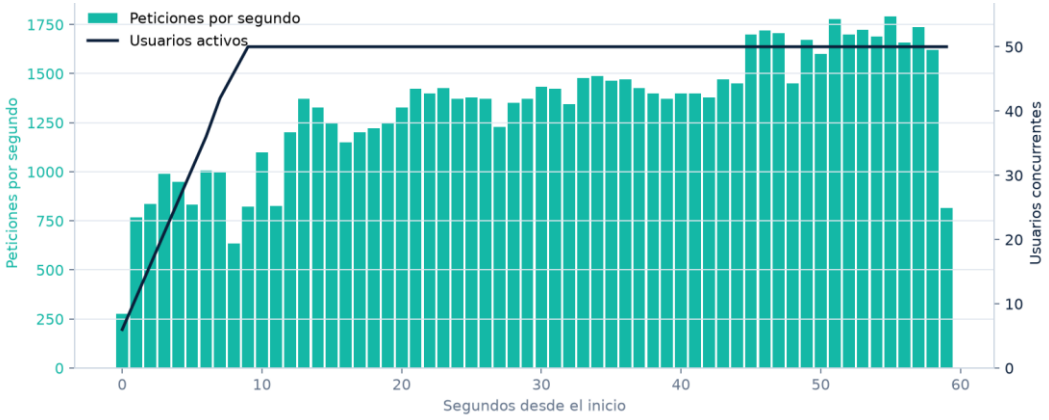


Figura 22. Throughput y usuarios concurrentes durante la prueba de carga con 50 usuarios.

Tabla 20. Resumen global de la prueba de estrés con 150 usuarios concurrentes.

Métrica	Resultado	Umbral	Estado
Muestras procesadas	113 600	Carga extrema	Informativo
Tasa de error	0.00 %	≤ 5.0 %	Cumple
Throughput	1 265.24 req/s	Alta concurrencia	Excelente
Promedio	108.9 ms	Informativo	Informativo
Mediana (p50)	101.0 ms	≤ 5000 ms	Cumple
p90	132.0 ms	≤ 5000 ms	Cumple
p95	151.0 ms	≤ 5000 ms	Cumple

Métrica	Resultado	Umbral	Estado
p99	381.0 ms	Informativo	Informativo
Máximo	1 072.0 ms	≤ 5000 ms	Cumple
Desviación estándar	47.9 ms	Informativo	Comportamiento predecible

Tabla 21**.** Detalle por transacción de la prueba de estrés.

Transacción	Muestras	Promedio	p95	p99	Throughput	Error
GET /api/equipment	28 421	108.9 ms	152.0 ms	380.0 ms	316.55 req/s	0.00 %
GET /api/equipment/stats	28 345	108.3 ms	150.0 ms	374.0 ms	315.70 req/s	0.00 %
GET /api/equipment?category=Cómputo	28 377	108.4 ms	150.0 ms	380.2 ms	316.06 req/s	0.00 %
GET /api/reservations/me	28 307	108.1 ms	150.0 ms	369.0 ms	315.28 req/s	0.00 %
POST /api/auth/login	150	461.2 ms	963.3 ms	1 033.4 ms	1.67 req/s	0.00 %
TOTAL	**113 600**	**108.9 ms**	**151.0 ms**	**381.0 ms**	**1 265.24 req/s**	**0.00 %**

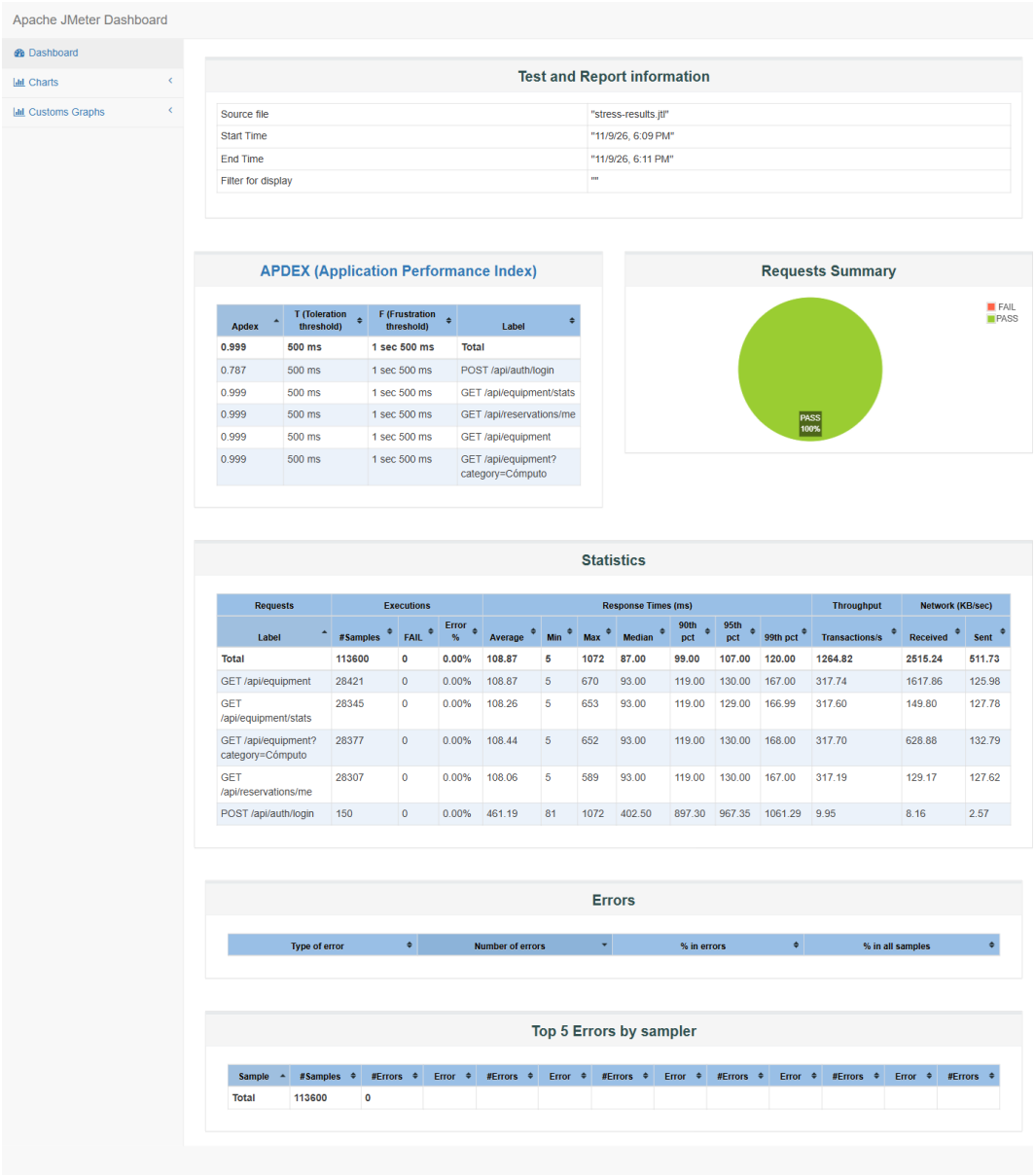


Figura 23**.** Dashboard de JMeter de la prueba de estrés: 0.00 % de error sobre 113 600 peticiones.

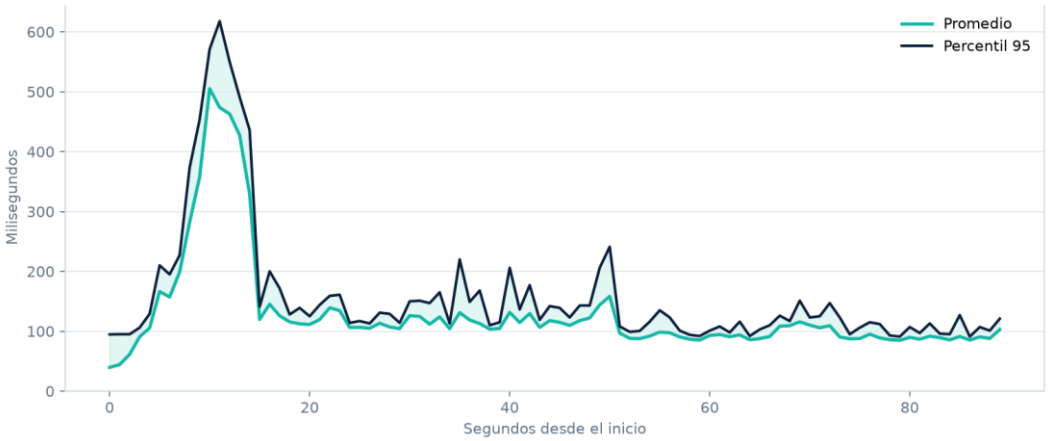


Figura 24. Tiempo de respuesta (promedio y percentil 95) durante la prueba de estrés con 150 usuarios.

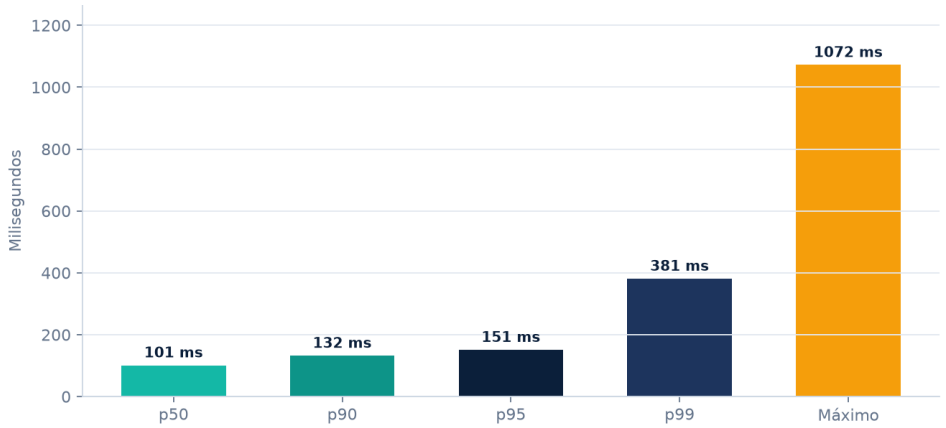


Figura 25. Distribución de percentiles de la prueba de estrés con 150 usuarios.

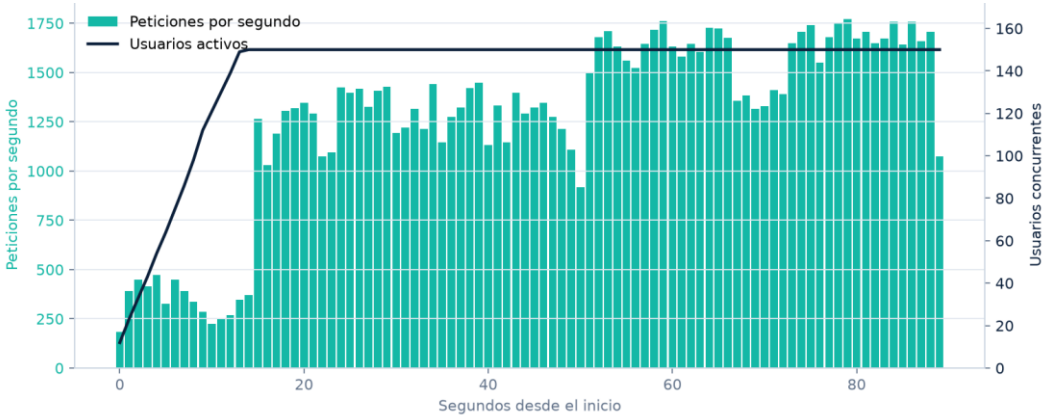


Figura 26. Throughput y usuarios concurrentes durante la prueba de estrés con 150 usuarios.

El incremento de latencia entre carga y estrés es coherente con el tripling de concurrencia: el p95 pasó de 48 ms a 151 ms y el máximo de 230 ms a 1 072 ms, sin errores ni caídas. La transacción de inicio de sesión es la más costosa porque implica bcrypt, un diseño deliberado para dificultar ataques de fuerza bruta. El sistema no colapsó en ninguno de los dos escenarios.

Accesibilidad e inclusión

La revisión se hizo con axe-core 4.10.2, el mismo motor de la extensión, sobre el frontend compilado en localhost:4173. Se evaluaron las páginas de inicio de sesión y catálogo con las reglas WCAG 2.0 y 2.1 de niveles A y AA. WAVE no se ejecutó, por lo que no se reportan sus resultados ni se simulan sus paneles.

Tabla 22**.** Resultados de la revisión automatizada de accesibilidad con axe-core.

Página	Reglas aprobadas	Violaciones	Elementos afectados	Revisión manual
Inicio de sesión	19	color-contrast (serious)	1	1
Catálogo	17	color-contrast (serious)	17	0

El único tipo de violación detectado es el contraste de color. No aparecieron faltas de nombre accesible, de etiquetas de formulario, de orden de encabezados ni de atributos ARIA inválidos, lo que sugiere que la refactorización de diálogos y avisos cumplió su objetivo. El contraste, sin embargo, afecta la legibilidad para personas con baja visión y debe corregirse antes de afirmar conformidad con WCAG 2.1 AA.

El defecto se registra como RF-14 en el backlog, con prioridad alta y estado pendiente. La corrección exige ajustar la paleta de los elementos señalados y volver a ejecutar axe-core hasta obtener cero violaciones. También conviene complementar la revisión automática con pruebas de teclado y con al menos una sesión de prueba con usuario real, porque axe-core solo cubre parte de los criterios.

RECOMENDACIONES FINALES

Conclusiones

La auditoría permite afirmar que LabTrack es un sistema funcionalmente correcto en el alcance que puede verificarse, con un código limpio y un desempeño sólido bajo concurrencia. No permite afirmar que esté completo respecto del alcance declarado ni que sea plenamente accesible. El veredicto honesto es «certificado con observaciones»: apto para operar en el alcance verificado, con brechas documentadas que deben cerrarse antes de una implantación institucional plena.

- Corrección funcional verificada: los siete flujos críticos de las cinco historias con evidencia pasan de extremo a extremo en un navegador real. La primera corrida había aprobado cuatro; las tres fallas restantes se explicaron por una regresión de selectores y se corrigieron en el código de pruebas.
- Salud del código: SonarQube pasó de 44 incidencias (1 bug, 2 vulnerabilidades, 41 code smells; confiabilidad B, seguridad C) a cero incidencias con calificaciones A/A/A. La deuda técnica estimada se redujo de 1 h 37 min a cero.
- Cobertura de pruebas: 101 casos aprobados, con 95.71 % de sentencias cubiertas en backend y 90.26 % en frontend. Las pruebas de límites añadidas elevan la confianza sobre las reglas de reserva.
- Desempeño: 0.00 % de errores en carga y estrés, con p95 de 48 ms y 151 ms. El sistema soporta el triple de la concurrencia nominal sin degradación catastrófica.
- Accesibilidad: la revisión automatizada solo encontró fallas de contraste, lo que confirma la mejora semántica de la refactorización, pero impide declarar conformidad WCAG 2.1 AA.
- Alcance: siete de las doce historias declaradas carecen de evidencia verificable. El sistema no debe presentarse como completo hasta que esas brechas se implementen y se prueben.

Recomendaciones

- Corregir el contraste de color señalado por axe-core y volver a ejecutar la revisión hasta obtener cero violaciones; complementarla con navegación por teclado y una prueba con usuario.
- Implementar las historias HU-06, HU-07, HU-08, HU-11, HU-13, HU-14 y HU-15, o bien reducir formalmente el alcance del proyecto para que coincida con lo verificable.

- Ajustar HU-01 y HU-05 a sus criterios declarados: validación estricta del código y persistencia del plazo de sanción de siete días.
- Incorporar la suite completa y el escaneo de SonarQube en integración continua, con bloqueo de la fusión cuando falle la puerta de calidad o alguna prueba.
- Inyectar las rutas LCOV en el scanner para que SonarQube refleje la cobertura real y no un 0.0 % engañoso.
- Reejecutar la suite E2E después de cualquier refactorización de accesibilidad o de marcado; la regresión S6819 demostró que un cambio correcto puede romper las pruebas.
- Validar formalmente los cinco casos sugeridos por IA antes de incorporarlos a la suite oficial, dejando registro del revisor y del resultado.
- Ejecutar Testomat.io o TestLink y WAVE cuando el entorno lo permita, y archivar las capturas correspondientes para completar la evidencia de herramientas.
- Parametrizar Selenium para Firefox y Edge y ampliar las pruebas de rendimiento con escenarios de red lenta y base de datos degradada.

El valor del trabajo no reside solo en las cifras favorables, sino en la trazabilidad de lo que se hizo y en la honestidad sobre lo que falta. Un informe que declare cero defectos y doce historias completas sería más agradable de leer y menos útil para tomar decisiones.

REFERENCIAS O BIBLIOGRAFÍA

American Psychological Association. (2020). Publication manual of the American Psychological Association (7th ed.). https://doi.org/10.1037/0000165-000

Apache Software Foundation. (2024). Apache JMeter user's manual (Version 5.6.3). https://jmeter.apache.org/usermanual/

Deque Systems. (2024). axe-core: Accessibility engine for automated web UI testing. https://github.com/dequelabs/axe-core

IEEE. (2008). IEEE standard for software and system test documentation (IEEE Std 829-2008). https://doi.org/10.1109/IEEESTD.2008.4578383

International Organization for Standardization. (2011). Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models (ISO/IEC 25010:2011). https://www.iso.org/standard/35733.html

International Software Testing Qualifications Board. (2023). Certified tester foundation level syllabus (Version 4.0). https://www.istqb.org/

NestJS. (2024). NestJS documentation: A progressive Node.js framework. https://docs.nestjs.com/

OWASP Foundation. (2021). OWASP Top 10:2021. https://owasp.org/Top10/

Prisma. (2024). Prisma ORM documentation. https://www.prisma.io/docs

React. (2024). React documentation. https://react.dev/

Selenium Project. (2024). Selenium WebDriver documentation. https://www.selenium.dev/documentation/

SonarSource. (2024). SonarQube documentation. https://docs.sonarsource.com/sonarqube/

Testomat.io. (2024). Testomat.io documentation. https://testomat.io/

Vitest. (2024). Vitest documentation. https://vitest.dev/

World Wide Web Consortium. (2018). Web Content Accessibility Guidelines (WCAG) 2.1. https://www.w3.org/TR/WCAG21/

World Wide Web Consortium. (2023). Accessible Rich Internet Applications (WAI-ARIA) 1.2. https://www.w3.org/TR/wai-aria-1.2/

ANEXOS – RECURSOS

Los anexos reúnen la matriz completa de casos, el registro del uso de agentes de inteligencia artificial, el índice de artefactos y la guía de reproducción. Se incluyen como evidencia auditable y como material de consulta para el equipo.

Anexo A. Matriz completa de casos de prueba y trazabilidad

La matriz se organiza por historia de usuario. Cada fila indica el identificador del caso, su origen (equipo o IA), la prioridad, el resultado esperado, la evidencia automatizada y el estado. La versión editable, con precondiciones y pasos detallados, está en docs/matriz-pruebas.md.

HU-01 · Autenticación Segura

ID	Caso de prueba	Origen	Pri.	Resultado esperado	Nivel y evidencia	Estado
CP-01	Inicio de sesión válido de estudiante	H	P1	HTTP 200, JWT emitido, usuario sin `passwordHash`; redirección a `/catalogo`	API `auth.spec.ts` · E2E `03-catalogo-estudiante.png`	Aprobado
CP-02	Inicio de sesión válido de administrador	H	P1	HTTP 200, rol `ADMIN`, acceso habilitado al panel de administración	API `auth.spec.ts`	Aprobado
CP-03	Credenciales inválidas muestran mensaje exigido	H	P1	HTTP 401 y mensaje exacto: ***"Credenciales inválidas."** en la interfaz	API `auth.spec.ts` · E2E `01-login-credenciales-invalidas.png`	Aprobado
CP-04	Usuario inexistente no revela existencia	IA	P2	HTTP 401 y mismo mensaje: ***"Credenciales inválidas."** (previene enumeración)	API `auth.spec.ts`	Aprobado
CP-05	Bloqueo temporal tras 3 intentos fallidos	H	P1	Primeros 2 intentos: 401; tercer intento: HTTP 423 `ACCOUNT_LOCKED` y alerta en pantalla	API `auth.spec.ts` · E2E `02-login-cuenta-bloqueada.png`	Aprobado
CP-06	Cuenta bloqueada rechaza incluso clave correcta	H	P1	HTTP 423; se mantiene `lockedUntil` en base de datos hasta cumplir el tiempo	API `auth.spec.ts`	Aprobado
CP-07	Validación de campos requeridos	IA	P2	HTTP 400 en API; validación visual de campos obligatorios en el cliente	API `auth.spec.ts` · UI `LoginPage.test.tsx`	Aprobado
CP-08	Normalización de espacios en código	IA	P3	Código saneado por DTO; autenticación exitosa sin errores	API `auth.spec.ts`	Aprobado
CP-09	Cierre de sesión (Logout)	H	P2	Token removido de localStorage; redirección inmediata a `/login`	API `auth.spec.ts` · UI `AuthContext`	Aprobado
CP-10	Estudiante restringido de rutas de administrador	H	P1	HTTP 403 Forbidden devuelto por los Guards de NestJS	API `auth.spec.ts` y `admin.spec.ts`	Aprobado
CP-11	Redirección de ruta protegida sin token	H	P1	Redirección automática hacia `/login` por `ProtectedRoute`	UI `ProtectedRoute.test.tsx`	Aprobado
CP-12	Petición API sin cabecera Bearer es denegada	IA	P2	HTTP 401 Unauthorized inmediato	API `auth.spec.ts`	Aprobado

HU-02 · Catálogo y Filtrado de Equipos

ID	Caso de prueba	Origen	Pri.	Resultado esperado	Nivel y evidencia	Estado
CP-13	Catálogo muestra nombre, código, categoría y estado	H	P1	Cada tarjeta muestra: nombre, código inventario, categoría y badge Disponible/Prestado	API `equipment.spec.ts` · UI `CatalogPage.test.tsx` · E2E	Aprobado
CP-14	Filtro exacto por categoría "Cómputo"	H	P1	Se despliegan exclusivamente equipos clasificados como `Cómputo`	API `equipment.spec.ts` · UI `CatalogPage.test.tsx`	Aprobado
CP-15	Filtro exacto por categoría "Electrónica"	H	P1	Se despliegan exclusivamente equipos clasificados como `Electrónica`	API `equipment.spec.ts` · E2E `04-filtro-electronica.png`	Aprobado
CP-16	Filtro exacto por categoría "Redes"	H	P1	Se despliegan exclusivamente equipos clasificados como `Redes`	API `equipment.spec.ts` · UI `CatalogPage.test.tsx`	Aprobado
CP-17	Mensaje exacto cuando categoría no tiene equipos	H	P1	Mensaje exacto: **"No hay equipos disponibles en esta categoría"**	UI `CatalogPage.test.tsx`	Aprobado
CP-18	Categoría desconocida devuelve lista completa	IA	P3	HTTP 200 y catálogo completo tolerante a fallos sin errores 500	API `equipment.spec.ts`	Aprobado
CP-19	Filtrado exclusivo de equipos disponibles	H	P2	Solo equipos en estado `AVAILABLE`; omite los que están `BORROWED`	API `equipment.spec.ts`	Aprobado
CP-20	Búsqueda por texto en tiempo real	IA	P2	Filtrado reactivo de tarjetas en el cliente coincidiendo por nombre o código	UI `CatalogPage.test.tsx`	Aprobado
CP-21	Métricas numéricas del catálogo	IA	P3	Conteo exacto de total, disponibles, prestados y en mantenimiento	API `equipment.spec.ts` · UI `CatalogPage.test.tsx`	Aprobado

ID	Caso de prueba	Origen	Pri.	Resultado esperado	Nivel y evidencia	Estado
CP-22	Consulta de equipo inexistente arroja 404	IA	P3	HTTP 404 con mensaje descriptivo de no encontrado	API `equipment.spec.ts`	Aprobado
CP-23	Catálogo requiere autenticación obligatoria	H	P2	HTTP 401 rechazando el acceso no autenticado	API `equipment.spec.ts`	Aprobado

HU-03 · Reserva de Equipos (Flujo Crítico)

ID	Caso de prueba	Origen	Pri.	Resultado esperado	Nivel y evidencia	Estado
CP-24	Registrar solicitud de reserva conforme	H	P1	HTTP 201 Created, estado `PENDING`, sin penalización	API `reservations.spec.ts` · E2E `05-reserva-registrada.png`	Aprobado
CP-25	Bloquear solapamiento con reserva aprobada	H	P1	HTTP 400 indicando cruce con reserva aprobada existente	API `reservations.spec.ts`	Aprobado
CP-26	Bloquear más de una reserva activa por estudiante	H	P1	HTTP 400: el estudiante ya mantiene una reserva activa	API `reservations.spec.ts`	Aprobado
CP-27	Reservas contiguas son admitidas sin cruce	IA	P2	HTTP 201 Created; los extremos iguales no se consideran cruce	API `reservations.spec.ts`	Aprobado
CP-28	Rechazar reserva sobre equipo prestado	H	P1	HTTP 400 advirtiendo que el equipo no está disponible	API `reservations.spec.ts`	Aprobado
CP-29	Rechazar solicitud con fecha pasada	IA	P2	HTTP 400 por validación de fecha retroactiva	API `reservations.spec.ts`	Aprobado
CP-30	Hora de inicio posterior o igual a fin	H	P1	HTTP 400; la interfaz alerta que el inicio debe ser anterior al fin	API `reservations.spec.ts` · UI `NewReservationPage.test.tsx`	Aprobado
CP-31	Restricción de duración máxima (4 horas)	H	P1	HTTP 400; bloqueo por regla de negocio de máximo 4 horas	API `reservations.spec.ts` · UI `NewReservationPage.test.tsx`	Aprobado
CP-32	Horario fuera de atención del laboratorio	IA	P2	HTTP 400; horario de atención permitido: 07:00 a 22:00	API `reservations.spec.ts`	Aprobado
CP-33	Formato de hora malformado en payload	IA	P2	HTTP 400 por validación de formato `HH:mm` de class-validator	API `reservations.spec.ts`	Aprobado
CP-34	Estudiante penalizado no puede reservar	H	P1	HTTP 400 denegando transacción por estado de cuenta sancionado	API `reservations.spec.ts`	Aprobado
CP-35	Administrador no puede solicitar préstamos	H	P2	HTTP 403; la funcionalidad de reserva es exclusiva de estudiantes	API `reservations.spec.ts`	Aprobado
CP-36	Reserva con ID de equipo inexistente	IA	P2	HTTP 404 indicando que el equipo no existe en catálogo	API `reservations.spec.ts`	Aprobado
CP-37	Estudiante únicamente visualiza sus propias reservas	H	P1	Retorna estrictamente las reservas del estudiante en sesión	API `reservations.spec.ts`	Aprobado
CP-38	Petición de reserva sin JWT es rechazada	H	P2	HTTP 401 Unauthorized	API `reservations.spec.ts`	Aprobado
CP-39	Preselección de equipo desde parámetro URL	IA	P2	El formulario precarga el equipo seleccionado en el catálogo	UI `NewReservationPage.test.tsx`	Aprobado
CP-40	Notificación de confirmación de solicitud	H	P2	Mensaje de éxito visible y redirección a "Mis Reservas"	UI `NewReservationPage.test.tsx` · E2E	Aprobado
CP-41	Selector de duración rápida (1h a 4h)	IA	P3	Calcula automáticamente hora de fin sumando 2 horas al inicio	UI `NewReservationPage.tsx`	Aprobado

HU-04 · Gestión de Solicitudes (Panel de Administrador)

ID	Caso de prueba	Origen	Pri.	Resultado esperado	Nivel y evidencia	Estado
CP-42	Listar solicitudes pendientes por defecto	H	P1	Retorna lista de solicitudes pendientes con datos de estudiante y equipo	API `admin.spec.ts` · UI `AdminPendingPage.test.tsx`	Aprobado
CP-43	Filtrado por estado administrativo	IA	P2	Filtrado correcto de registros según el parámetro enviado	API `admin.spec.ts`	Aprobado
CP-44	Aprobar reserva y marcar equipo como prestado	H	P1	Estado `APPROVED`, equipo pasa a `BORROWED`, evento en auditoría	API `admin.spec.ts` · E2E `06-solicitud-aprobada.png`	Aprobado
CP-45	Impedir aprobar si se genera cruce con otra aprobada	H	P1	HTTP 400 por cruce de horario con otra reserva previamente aprobada	API `admin.spec.ts`	Aprobado
CP-46	Solo es posible aprobar solicitudes pendientes	IA	P2	HTTP 400 indicando que la reserva ya no está en estado pendiente	API `admin.spec.ts`	Aprobado
CP-47	Rechazo exige motivo obligatorio (mínimo 5 letras)	H	P1	La interfaz bloquea el envío y alerta campo requerido	API `admin.spec.ts` · UI `AdminPendingPage.test.tsx`	Aprobado

ID	Caso de prueba	Origen	Pri.	Resultado esperado	Nivel y evidencia	Estado
CP-48	Motivo de rechazo demasiado corto	IA	P2	HTTP 400 de validación; la solicitud permanece en `PENDING`	API `admin.spec.ts`	Aprobado
CP-49	Rechazo persiste motivo visible para el estudiante	H	P1	Estado `REJECTED`, motivo guardado y visible en "Mis Reservas"	API `admin.spec.ts` · UI `MyReservationsPage.test.tsx`	Aprobado
CP-50	Bloqueo de acceso de estudiantes a administración	H	P1	HTTP 403 Forbidden	API `admin.spec.ts`	Aprobado
CP-51	Indicadores administrativos de laboratorio	IA	P2	Conteo exacto de reservas activas, préstamos y usuarios sancionados	API `admin.spec.ts` · UI `AdminPendingPage.test.tsx`	Aprobado
CP-52	Bitácora inmutable de auditoría	H	P2	Registro de tipo de evento, código de actor, fecha e IP	API `admin.spec.ts`	Aprobado
CP-53	Manejo de aprobación sobre reserva inexistente	IA	P3	HTTP 404 Not Found con mensaje claro	API `admin.spec.ts`	Aprobado

HU-05 · Devolución y Penalización Automática

ID	Caso de prueba	Origen	Pri.	Resultado esperado	Nivel y evidencia	Estado
CP-54	Devolución puntual sin penalización	H	P1	`penaltyApplied = false`, estudiante continúa `ACTIVE`, equipo `AVAILABLE`	API `admin.spec.ts`	Aprobado
CP-55	Devolución tardía penaliza automáticamente	H	P1	`penaltyApplied = true`, estudiante cambia a `PENALIZED` automáticamente	API `admin.spec.ts` · E2E `07-devolucion-penalizada.png`	Aprobado
CP-56	Equipo devuelto retorna a estado Disponible	H	P1	El equipo cambia inmediatamente su estado de `BORROWED` a `AVAILABLE`	API `admin.spec.ts`	Aprobado
CP-57	Solo se procesan devoluciones de reservas aprobadas	IA	P2	HTTP 400 advirtiendo que solo aplica a reservas aprobadas	API `admin.spec.ts`	Aprobado
CP-58	Registro de evento de penalización en bitácora	H	P1	Evento `PENALTY_APPLIED` registrado con código de infractor	API `admin.spec.ts` · E2E `08-bitacora-penalizacion.png`	Aprobado
CP-59	Banner de advertencia de penalización al estudiante	IA	P2	Banner visible: **"Tu cuenta está penalizada por entrega tardía"**	UI `MyReservationsPage.test.tsx`	Aprobado
CP-60	Alerta previa al encargado antes de sancionar	IA	P2	Mensaje modal informa que la acción penalizará al estudiante	UI `AdminReturnsPage.test.tsx`	Aprobado
CP-61	Indicador de tiempo transcurrido desde vencimiento	H	P3	Etiqueta dinámica: `"Vencida hace X h Y min"` para priorizar	UI `AdminReturnsPage.test.tsx`	Aprobado

Anexo B. Uso de agentes de IA: prompts y skills

El trabajo se desarrolló con un equipo de agentes de inteligencia artificial coordinado desde la línea de comandos. Un agente líder dividió el trabajo en tareas con alcances de archivos que no se solapan; cada agente trabajó en su propia copia aislada del repositorio y el líder revisó el reporte y el diff antes de integrar. Este anexo deja constancia de qué se le pidió a cada agente y de las intervenciones humanas que fueron necesarias. El documento completo está en docs/prompts-y-skills-agentes.md.

Skills y mecanismos utilizados

Tabla 23**.** Skills y mecanismos empleados durante el desarrollo asistido.

Skill o mecanismo	Quién la usó	Para qué
multi-cli-team (invocada con /team)	Agente líder	Orquestar el equipo: planificación, ejecución, recolección y fusión mediante team.ps1.
Herramientas MCP de Playwright	Agente líder	Capturas reales de SonarQube y de los reportes de JMeter; ejecución de axe-core.
sonar-scanner, SonarQube, JMeter, Vitest, Selenium	Líder y agentes	Análisis estático, carga y estrés, pruebas unitarias y flujos E2E.

Los agentes de trabajo podían cargar sus propias skills y servidores MCP; no se registró cuáles usó cada uno, por lo que no se afirma nada al respecto.

Reparto de agentes

Tabla 24**.** Tareas asignadas a cada agente y alcance de archivos.

Tarea	Agente	Alcance	Resultado
sonar-antes-despues	opencode	docs/evidencias-sonarqube/**	Reprodujo el ANTES y generó JSON y paneles de apoyo; la ejecución real la completó el líder.
pruebas-unitarias	copilot	docs/evidencias-pruebas/**, pruebas .extra	Ejecutó y documentó las pruebas; agregó dos archivos con ocho casos nuevos.
jmeter-carga-estres	agy	jmeter/**, docs/evidencias-jmeter/**	Dejó plan de estrés y análisis; la ejecución real la hizo el líder tras instalar JMeter.
informe-presentacion-backlog	agy	docs/*.py, *.docx, *.pdf, *.pptx, *.md, assets/**	Informe, presentación y backlog del primer intento.
historias-usuario	copilot	docs/generate_*.py, *.md, *.docx, *.pdf, *.pptx	Verificó cada historia contra el código y actualizó los entregables.

Decisión del usuario: la redacción del informe y de la presentación no debía realizarla el agente líder; se asignó a agy y luego la verificación de historias a copilot. El agente dsh no estaba instalado y no se utilizó.

Prompts enviados a cada agente

Los prompts se transcriben tal como figuran en el registro del equipo. En el primer envío se pedían además evidencias con capturas del ANTES y del DESPUÉS.

sonar-antes-despues — agente opencode

Objetivo: evidencias visuales de SonarQube del ANTES y DESPUÉS de LabTrack. No existe versión ANTES en Git (solo un commit). Crear en docs/evidencias-sonarqube/antes/src/

una copia de backend/src y frontend/src y reintroducir los defectos de sonar-baseline.json (S3358, S1082, S6819, S6847, S2068, S2245, S6759 y el resto), con los mismos archivos y líneas. Todo lo reconstruido debe etiquetarse como ANTES REPRODUCIDO, nunca como captura histórica. No modificar backend/src ni frontend/src reales. Para el DESPUÉS analizar el código actual con projectKey labtrack-despues. Guardar capturas legibles de dashboard, bugs, vulnerabilidades, code smells, ratings, quality gate y listado de issues, además de JSON de la API, logs sin tokens, tabla comparativa y gráfico. (El registro del prompt está truncado en el documento fuente.)

pruebas-unitarias — agente copilot

Contexto: backend NestJS con Vitest y frontend React con Vitest. Ejecutar las pruebas y la cobertura de ambos, guardar salidas y capturas. Documentar el estado antes y después de agregar casos. Agregar pruebas nuevas solo como archivos nuevos terminados en .extra, cubriendo casos límite. Incluir al menos dos pruebas que demuestren falla ante un defecto simulado en un mock local, sin editar backend/src ni frontend/src. Escribir la matriz de casos con pruebas unitarias, funcionales, referencia a JMeter y casos sugeridos por IA con validación pendiente. Agregar plantillas para Katalon y Testomat.io/TestLink y el CSV importable, sin inventar capturas de herramientas no ejecutadas. No hacer commit.

jmeter-carga-estres — agente agy

Levantar el backend en un puerto local con base de pruebas y ejecutar una prueba de carga y una de estrés con JMeter. Si JMeter no está instalado, reportarlo y dejar los comandos exactos; no inventar resultados. Guardar .jtl crudos, reporte HTML, tablas de muestras, promedios, percentiles, throughput y porcentaje de error, gráficas y capturas del reporte. Escribir un análisis en español con contexto sobre carga, estrés y percentiles. No tocar backend/src ni frontend/src. Apagar el backend al terminar. No hacer commit.

informe-presentacion-backlog — agente agy

Redactar el informe y la presentación usando solo cifras que salgan de los archivos de evidencia. Respetar los hechos: SonarQube 44 incidencias antes y 0 después, con el ANTES reproducido; cobertura de SonarQube en 0.0 % por falta de LCOV y cobertura real de Vitest de 95.71 % y 90.26 %; JMeter con 50 y 150 usuarios; pruebas unitarias de 93 a 101; Katalon y Testomat.io/TestLink no ejecutados; casos de IA con validación pendiente. Incrustar las capturas reales y no usar los paneles reconstruidos. Actualizar el backlog y la matriz. Ejecutar los generadores y verificar los productos. No hacer commit.

historias-usuario — agente copilot

Para cada historia declarada, verificar en backend/src, frontend/src, backend/test y frontend/src/test si existe código y pruebas, y clasificarla como implementada con pruebas, implementada sin pruebas o declarada sin evidencia verificable. No afirmar implementación ni pruebas que no se encuentren. Crear la trazabilidad, actualizar el informe, la matriz, el backlog y

la presentación sin superar las cinco diapositivas. Regenerar los entregables y verificar. No modificar backend/src, frontend/src ni docs/evidencias-*. No hacer commit.

Intervenciones del líder que los agentes no realizaron:

- Instaló y ejecutó SonarQube, sonar-scanner, JDK 21 y JMeter, herramientas que los agentes no tenían disponibles.
- Detectó y corrigió falsos positivos del ANTES y un escaneo del DESPUÉS contaminado por configuraciones de carpetas de trabajo.
- Ejecutó Selenium y encontró la regresión causada por la refactorización S6819.
- Ejecutó axe-core para la revisión de accesibilidad.
- Verificó que no se filtraran tokens al repositorio y resolvió el fallo de identidad de Git local.

Anexo C. Índice de artefactos y evidencias

Todos los archivos citados están presentes en el repositorio y forman parte del inventario auditable. La columna de naturaleza distingue lo real, lo reproducido, lo planificado y lo no ejecutado.

Tabla 25. Artefactos, ubicación y naturaleza de la evidencia.

Artefacto	Ubicación	Naturaleza	Contenido
Capturas SonarQube antes	docs/evidencias-sonarqube/antes/capturas-reales/	Real (estado reproducido)	Siete capturas con 44 incidencias
Capturas SonarQube después	docs/evidencias-sonarqube/despues/capturas-reales/	Real	Siete capturas con 0 incidencias
Comparativo SonarQube	docs/evidencias-sonarqube/comparacion/real/	Real	Tabla y gráfico de evolución
Código del ANTES	docs/evidencias-sonarqube/antes/src/	Reproducido	Copia con los defectos reinyectados
Salidas de pruebas	docs/evidencias-pruebas/capturas/	Real	Consolas de Vitest y de Selenium
Reportes de cobertura	docs/evidencias-pruebas/unitarias/cobertura/	Real	Resúmenes y reporte HTML
Selenium corrida 1	docs/evidencias-pruebas/selenium/corrida-1-regresion-detectada/	Real	JUnit XML con 3 fallos y capturas
Selenium corrida final	docs/evidencias-pruebas/selenium/corrida-3-final-aprobada/	Real	JUnit XML sin fallos y capturas 01–08
JMeter carga	docs/evidencias-jmeter/carga/	Real	Dashboard, summary.json, tablas y gráficos
JMeter estrés	docs/evidencias-jmeter/estres/	Real	Dashboard, summary.json, tablas y gráficos
Accesibilidad	docs/evidencias-accesibilidad/axe-resultados.json	Real	Violaciones de contraste por página
Plantilla Testomat.io	docs/evidencias-pruebas/testomatio-import.csv	Plantilla	Importación de casos, no ejecutada
Trazabilidad	docs/trazabilidad-historias-usuario.md	Documento	Verificación de las doce historias
Backlog	docs/refactoring-backlog.md	Documento	Catorce ítems técnicos y nueve brechas
Matriz de pruebas	docs/matriz-pruebas.md	Documento	66 casos funcionales y 10 de rendimiento

Anexo D. Guía de reproducción y credenciales de demostración

Los siguientes comandos permiten reproducir las evaluaciones en una máquina con Node.js, Java y Python instalados. Las rutas son relativas a la raíz del repositorio.

```
# 1. Base de datos y datos sembrados
cd backend
npm install
npm run db:setup

# 2. Pruebas unitarias y cobertura
npm run test:cov
cd ..\frontend
npm install
npm run test:cov

# 3. Verificación de tipos
npx tsc --noEmit
cd ..\backend
npx tsc --noEmit

# 4. Pruebas E2E con Selenium (backend en 3000, frontend en 4173)
cd ..\frontend
npm run test:e2e

# 5. Carga y estrés con JMeter
cd ..\jmeter
powershell -ExecutionPolicy Bypass -File .\run-load-test.ps1
powershell -ExecutionPolicy Bypass -File .\run-stress-test.ps1

# 6. Generación de figuras y entregables
cd ..\docs
python make_figures.py
python generate_report.py
python generate_presentation.py
```

Credenciales deterministas de demostración: estudiante con código 20240001 y administrador con código ADMIN01. Las contraseñas se obtienen del archivo de entorno o del seed de desarrollo y no se reproducen en este informe. Después de ejecutar pruebas E2E conviene restaurar la base con npm run db:reset en backend para recuperar el estado inicial.

NOTAS ACLARATORIAS

- Formato: el informe parte de la plantilla institucional, conserva su portada con los logotipos, el encabezado y el pie, y aplica Arial 11, texto justificado, interlineado 1,5, papel A4 y citas en APA 7.
- Índices: el índice de contenido, el de tablas, el de figuras y el de anexos se generan con campos de Word a partir de los títulos y de la numeración SEQ. Si el documento se abre en un procesador que no actualiza campos, deben actualizarse manualmente con F9.
- Cifras: todas las cifras provienen de archivos del repositorio. No se estimaron ni se redondearon valores para mejorar la presentación.
- Estado ANTES: el código previo a la refactorización no existe en el historial de Git; fue reproducido con los defectos documentados. Las capturas corresponden a esa ejecución real y se identifican como estado reproducido.
- Evidencias no realizadas: Testomat.io, TestLink, Katalon Studio y WAVE no se ejecutaron. Se entregan plantillas y se declara la limitación en lugar de simular capturas.
- Casos de IA: los cinco casos sugeridos por inteligencia artificial permanecen con validación humana pendiente y no se contabilizan como aprobados.
- Historias de usuario: siete de las doce declaradas no tienen evidencia verificable. El informe las reporta como brechas y no como funcionalidades terminadas.
- Marcadores pendientes: los textos resaltados con la expresión «COMPLETAR POR EL EQUIPO» indican información que el equipo debe aportar. Se listan en docs/pendientes-del-equipo.md.
- Las referencias se limitan a documentación oficial y estándares públicos verificables; no se incluyeron fuentes que el equipo no hubiera consultado.